

Introduction to Linux, Ubuntu and Shell Scripting

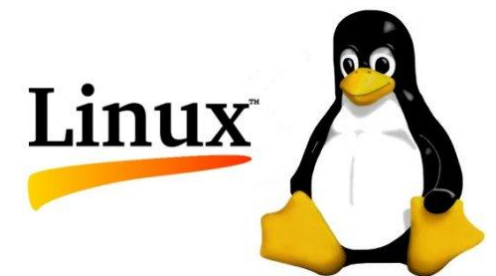
based on online slides by *CYBERPATRIOT*, www.uscyberpatriot.org and other internet resources

SECTION 1

What is Linux?

A Family of Operating Systems

- Linux refers to a family of operating systems modeled off of Unix
- Can perform many of the same functions as Windows or OS X
- Built in a collaborative, open-source environment
 - Anyone may use, modify, or distribute the Linux kernel
 - Anyone can develop software to run on the Linux kernel
 - Many programmers collaborate to develop or improve Linux programs
 - Many Linux operating systems and add-on programs are free

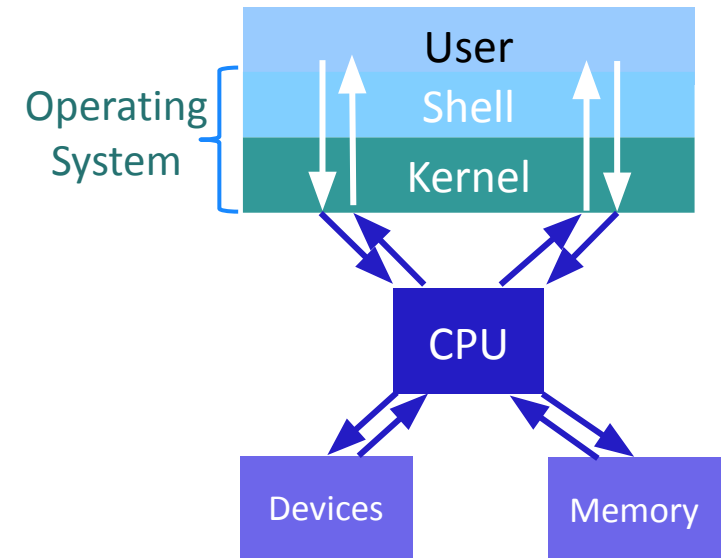


Source: <http://scienceblogs.com/gregladen/category/technology/linux/>

Sources: <http://www.linuxfederation.com/linux-everywhere/>, <http://techland.time.com/2012/06/19/what-exactly-is-a-supercomputer/>, <http://www.unixmen.com/why-do-super-computers-use-linux/>

Linux Kernel

- A kernel is the core component of an OS
 - Windows operating systems have kernels, but since they are not open-source or packaged separately for programmers to build off, they are less-often discussed
- Manages system resources (Memory, Processes, Input and Output Devices)
- When a user does something in the shell (the OS's user interface and applications), the kernel translates the command and prioritizes it against other requests for resources, so that it can be understood and executed by the CPU



Source: <https://help.ubuntu.com/8.04/serverguide/C/user-management.html>

Different Linux Operating Systems

- There are many different **flavors** (OSes) built off the Linux kernel
 - **Ubuntu**: Most popular flavor. It is free and is the most user-friendly.
 - **Mint**: A popular variation of Linux.
 - **Red Hat**: Designed by a company that develops specialized flavors for government and big business.
 - **Fedora**: An open-source, free version of Red Hat. Used frequently as a test bed for Red Hat programs.
- These **flavors** are similar at the basic level but can have very different interfaces and specialized commands.



Source: <http://www.ubuntu.com/>



Source: <http://www.linuxmint.com/>



Source: <https://fedoraproject.org/>



Source: <http://www.redhat.com/>

Sources: <http://distrowatch.com/dwres.php?resource=major>, <http://www.sitepoint.com/unix-style-operating-systems/>

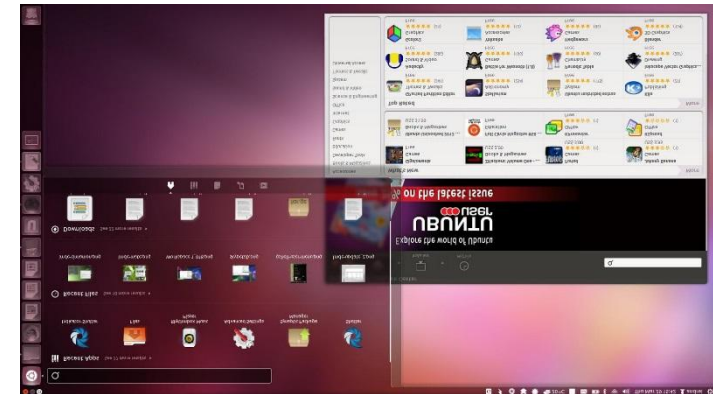
Differences Compared To Windows

- Often free or less expensive
- **Desktop environment** and **GUI** elements change
- Some tasks can only be run in the **command line**
- Less malware on Linux
- Certain hardware cannot work with Linux



Source:

<http://news.softpedia.com/newsImage/How-To-Install-KDE-4-1-On-Ubuntu-8-04-2.jpg/>

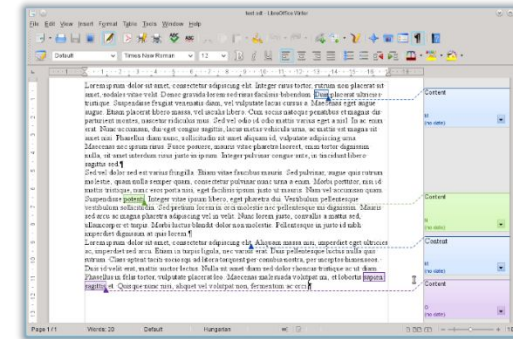


Source: <https://tekdrops.files.wordpress.com/2012/06/unity.jpeg>

Similarities to Windows

- Can be servers and workstations
 - Linux servers and workstations are more similar than Windows ones. Linux servers come pre-installed with server applications
- Can complete similar tasks
 - There are Linux programs that function like to Microsoft Office (LibreOffice), Outlook (Thunderbird), etc.
- Are stable and have significant support
- Subject to very similar vulnerabilities
 - Linux systems are targeted less frequently by malware, but still have many of the same vulnerabilities and patches (firewalls, password policy, etc.)

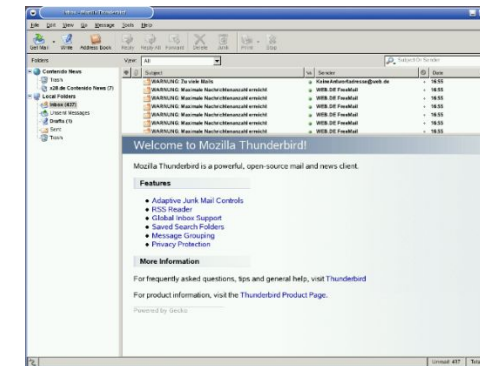
LibreOffice



Source:

http://es.libreoffice.org/assets/Uploads/EN-Project_images/4.0NewFeatures/Writer/Comment-text-range.png?r=45758

Mozilla Thunderbird



Source: http://commons.wikimedia.org/wiki/File:Mozilla_thunderbird_empty_screenshot.png

Sources: https://help.ubuntu.com/community/ServerFaq#What.27s_the_difference_between_desktop_and_server3F,
<http://blog.computerlagoon.com/2013/04/07/similarities-of-windows-and-linux/>

SECTION 2

Ubuntu Terminology and Concepts

The Root Account

- Account types: **User** and **Root**
- **Root** – The Linux Administrator account
 - Like the built-in Administrator in Windows, Linux comes with a built-in root account
 - A system can have multiple root accounts
 - Users can switch whether their actions are carried out as a user or root
 - When someone enacts root permissions, he or she can access all the files and run all commands on a system, as well as set policies for other users
- Root actions require a password in both GUI and command line
- **Authentication vs. Authorization**
 - **Root** users are **authorized** to do many different tasks, but they must first **authenticate** their identity by entering their **password**

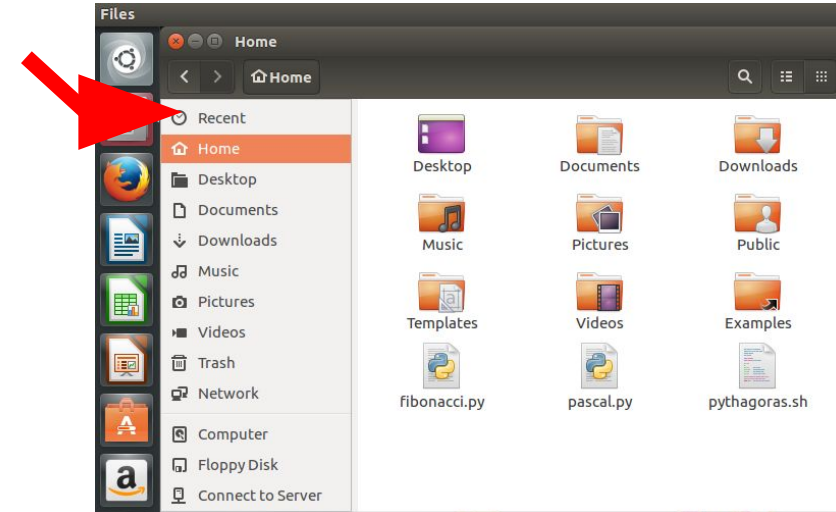


Source: <http://eswalls.com/wp-content/uploads/2014/01/i-am-root.png>

Source: <http://www.cyberciti.biz/faq/authentication-vs-authorization/>

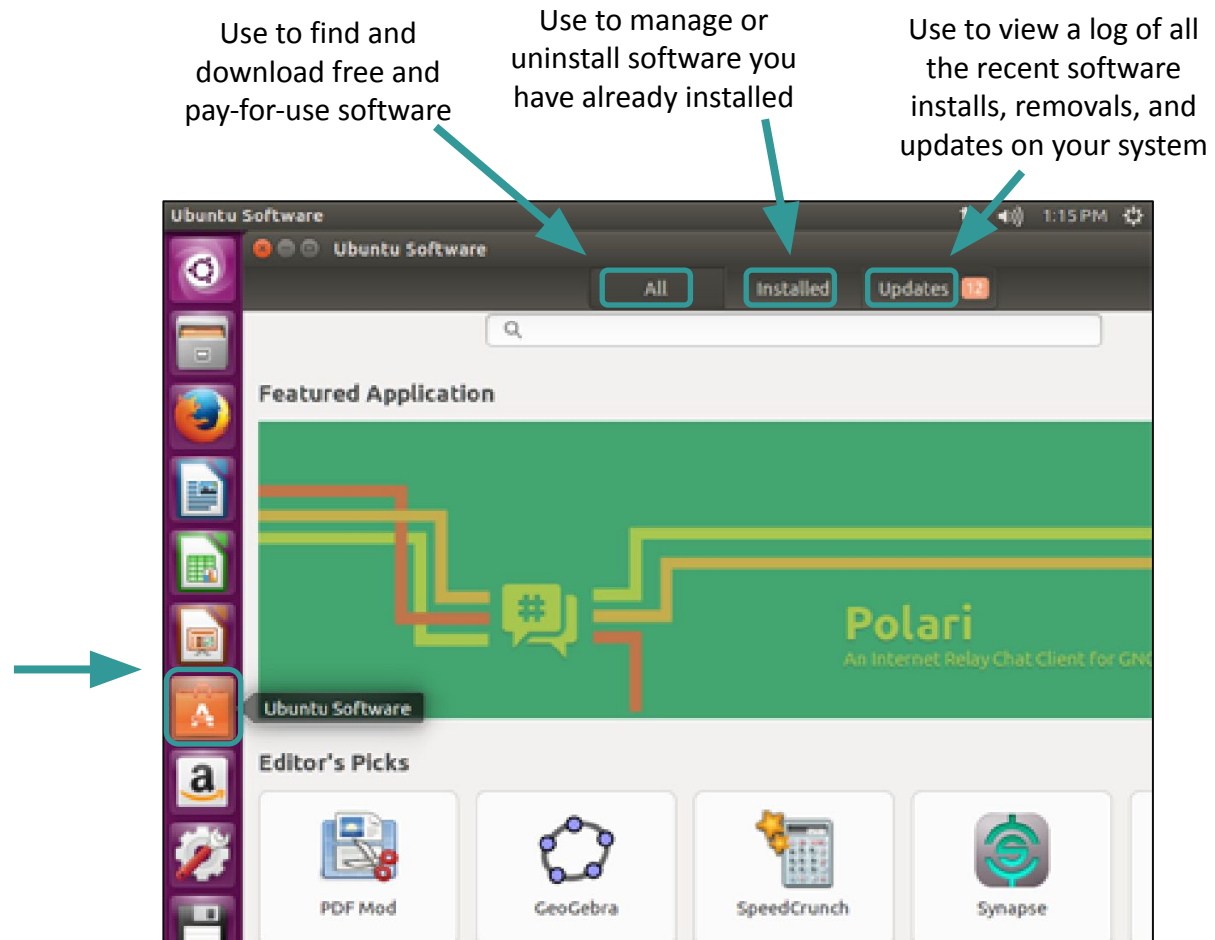
Ubuntu File System

- Different from the Windows file system
 - Does not specify on which drive a folder is stored and uses forward slashes (/) to identify root directories
- Example:
 - **Windows:** C:\Documents\hello.txt
 - **Linux:** /home/CyberPatriot/hello.txt
- Important folders:
 - **/home:** stores each user's documents, media files, etc. Users can only access their own folders, unless they have enacted root permissions.
 - **/boot:** contains startup files and kernel files. Should not be modified unless you are an expert user.
- The file system can be accessed by clicking the file cabinet on your Ubuntu menu bar



Adding and Removing Software

- Linux software is bundled into **packages**
- Packages are managed by **package managers**
 - In Ubuntu, the package manager is called “Ubuntu Software Center.”
 - It looks and functions a lot like Mac’s App Store
 - Many programs are free
- To access Ubuntu Software Center, **click the shopping bag on your Ubuntu menu bar**
- Users must enact root permissions to install, uninstall, or modify software



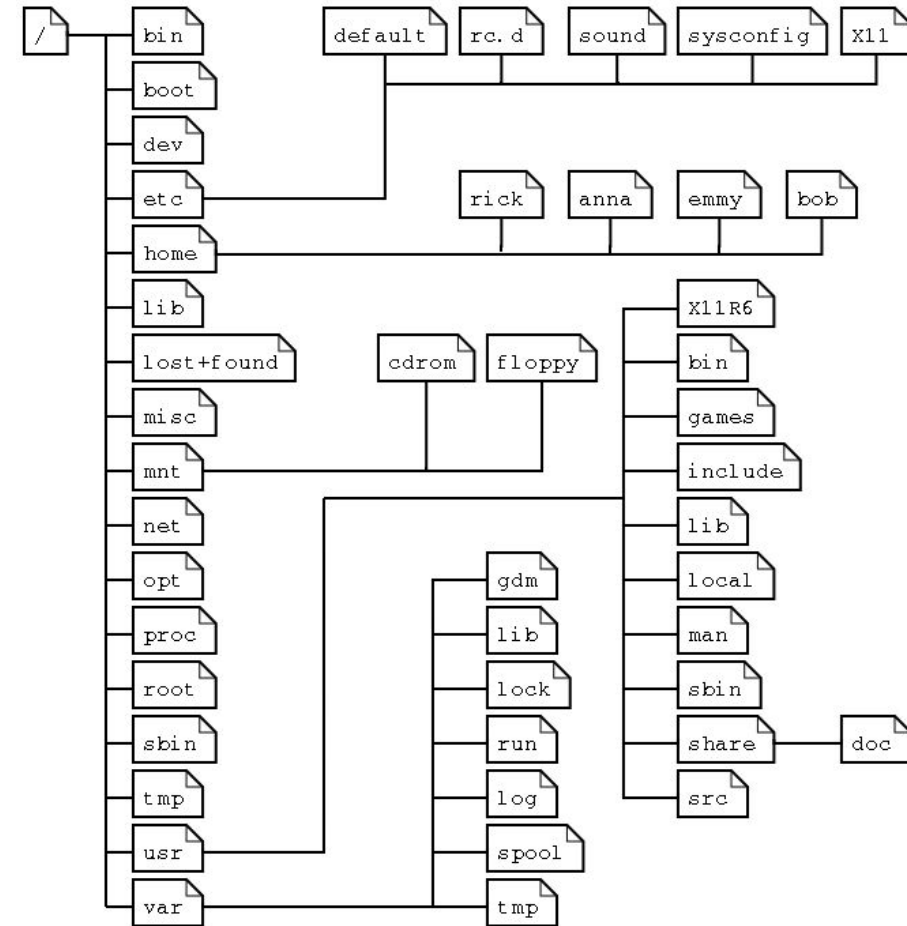
Source: <https://help.ubuntu.com/community/InstallingSoftware>

SECTION 3

Introduction to Ubuntu Command Line

Linux Filesystem

- Linux files are stored in **directories**, which are the same as **folders** in windows
- Linux filesystem tree
 - Base or trunk of the tree is the *root directory (/)*
 - Branches of the tree are *directories*
 - Leaves of the tree are *files*
- Linux commands, files, and directory names are **case sensitive**



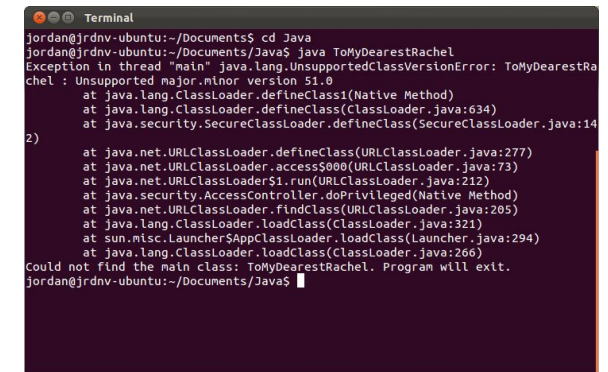
Command Line Pros and Cons

Pros

- Provides the user more control
 - Unlike the GUI, which pre-programs certain tasks, command line allows you to send more detailed and customized commands
- Only option for some tasks in Ubuntu
- Saves clicking time because it just requires a keyboard
- Uses less of the computer's processing power than the GUI (no animations or graphical processing)
- Can be made easier with scripting
 - **Scripts** are sequenced lists of commands that allow users to send multiple commands at once
 - Can be used for routine tasks like backing up files, monitoring system, and quickly gathering information about memory and processes

Cons

- Not as user-friendly as GUI
 - Requires memorizing commands or using a reference
- Harder to multitask
 - Having multiple command line windows open at once can be confusing, since they look nearly identical



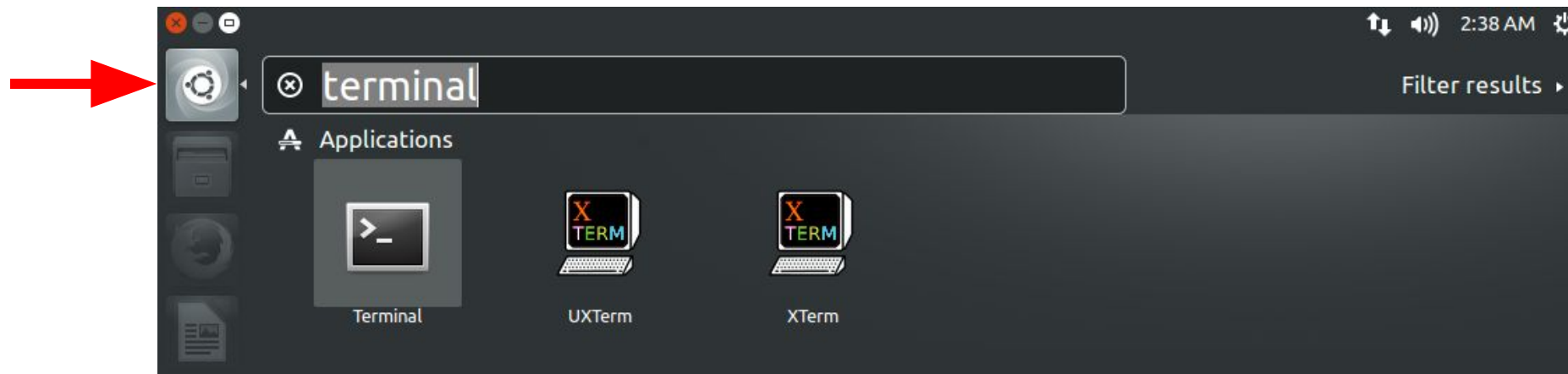
```
Terminal
jordan@jrdnv-ubuntu:~/documents$ cd Java
jordan@jrdnv-ubuntu:~/documents/Java$ java ToMyDearestRachel
Exception in thread "main" java.lang.UnsupportedClassVersionError: ToMyDearestRachel : Unsupported major.minor version 51.0
    at java.lang.ClassLoader.defineClass1(Native Method)
    at java.lang.ClassLoader.defineClass(ClassLoader.java:634)
    at java.security.SecureClassLoader.defineClass(SecureClassLoader.java:142)
    at java.net.URLClassLoader.defineClass(URLClassLoader.java:277)
    at java.net.URLClassLoader.access$000(URLClassLoader.java:73)
    at java.net.URLClassLoader$1.run(URLClassLoader.java:212)
    at java.security.AccessController.doPrivileged(Native Method)
    at java.net.URLClassLoader.findClass(URLClassLoader.java:205)
    at java.lang.ClassLoader.loadClass(ClassLoader.java:321)
    at sun.misc.Launcher$AppClassLoader.loadClass(Launcher.java:294)
    at java.lang.ClassLoader.loadClass(ClassLoader.java:266)
Could not find the main class: ToMyDearestRachel. Program will exit.
jordan@jrdnv-ubuntu:~/documents/Java$
```

Source: <http://i.stack.imgur.com/2hBjf.png>

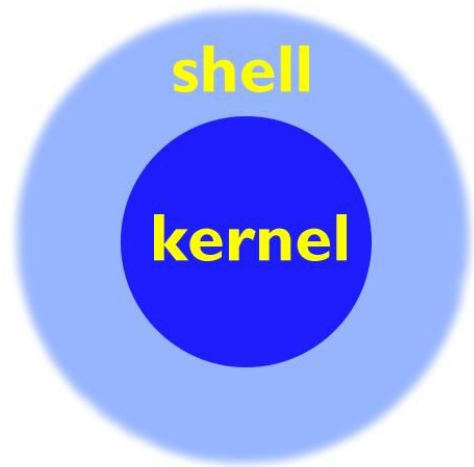
Source: <http://www.computerhope.com/issues/ch000619.htm>

Accessing The Command Line

- Terminal is the tool used to access the Ubuntu Command Line
 - Click the Ubuntu button
 - Type terminal
 - Press Enter or click the icon labeled Terminal
- OR
- Press Ctrl-Alt-T



Your Shell



- On user log-in, the system runs a *shell*
- A shell is the environment within which you will interface with the kernel via commands
- It determines the syntax for complex command-line operations and shell scripting
- The shell you're using is called "bash," the successor to the venerable "Bourne Shell" called "sh"
- BASH: "Bourne Again Shell"

Various Shells



- sh – the original UNIX shell
- bash – written as a replacement/extension of sh
- csh – C shell based on the C programming language developed in the late 1970s
- tcsh – enhanced version of C shell
- ksh – Korn shell developed in the early 1980's, backward compatible with sh, but with some features of csh
- zsh – extended version of sh, developed in 1990
- dash – developed as replacement for ash in Debian

Using Terminal

- When typing commands in **Terminal**, it is very important to pay attention to capitalization and spaces
- Hitting **Enter** will execute your command and hitting **Ctrl+D** will close any commands you have running or exit the Terminal
- There are numerous Ubuntu command databases and command line tutorials online. Here are a few sites:
 - <https://help.ubuntu.com/community/UsingTheTerminal>
 - <http://ryanstutorials.net/linuxtutorial/>
 - <http://manpages.ubuntu.com/>
 - <http://ubuntu-manual.org/>

Summary of Linux commands

awk	allows manipulation of text	more	scroll through file a page at a time
bg	place suspended job into background	mv	change the name of a file (move)
cal	display calendar	nano/pico	text editors
cat	view contents of a file	printenv	display shell variables
cd	change directory	ps	show current process information
chmod	change permissions on a file/directory	pwd	print current working directory
cp	copy a file	rm	delete or remove a file
cut	extract a field of data from text output	rmdir	delete or remove a directory
diff	compare files line by line	sed	stream editor
echo	output text to the terminal or to a file	sleep	pause
emacs	text editor	sort	perform a sort of text
fg	bring suspended job to foreground	tail	view end of the file
file	display file type	touch	create an empty file or update timestamps
find	search for files	tr	character substitution tool
grep	search a file or command output for a pattern	uniq	remove identical, adjacent lines
head	view beginning of file	vi/vim	text editor
history	display list of most recent commands	wc	print number of lines, words or characters
less	scroll forward or back through a file	which	shows full path of a command
ln	create a link to a file	whoami	displays username
ls	list files in a directory		
man	view information about a command		
mkdir	make directory		

Our cast! (of characters)

Character	Name/Location
\	Backslash (above the enter key)
/	Slash (left of right shift key)
`	Back-tick (left of the number 1, above the tab key)
	Pipe (shift-\)
[and]	Brackets (left of the backslash)
{ and }	Braces or “curly” brackets (shift-[and shift-])
< and >	Angle brackets (left of the right shift key)
~	Tilde (shift-~)
! , @ , # , \$, % , ^ , & , * , (,)	(!) Bang/exclamation mark, (@) at sign, (#) hash, (\$) dollar/string, (%) percent, (^) caret, (&) ampersand, (*) asterisk/start, and the left and right parenthesis.

Linux Command Basics

- Linux commands are case-sensitive

ls is not the same as LS

- Linux commands may have options that come after the command that start with a “-” and followed by a letter or “- -” and a word:

```
$ ls -r
```

```
$ ls --reverse
```

- Linux commands may allow for arguments:

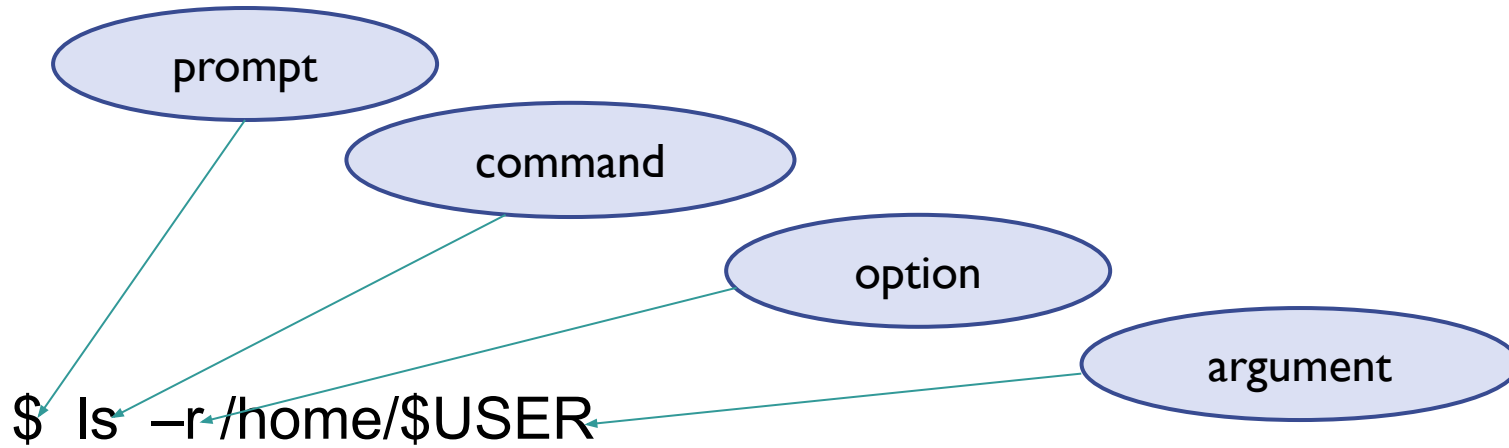
```
$ ls /tmp
```

- You can run more than one command on the same line by separating the commands with a semicolon (;)

```
$ ls;date
```

- Most Linux commands have a manual page or help to describe how they can be used....more about this later!

Linux Command Example



Command Syntax



- **Command:** tells the computer what you want it to do
 - All other components of the syntax depend on what the command is
 - The “cat” command creates, displays, or copies files
- **Option:** customizes the output of the command
 - “-n” told the computer to add a number to each line of text in the file you created
 - The effect an option has varies by command
 - Not required for all commands
- **Operator:** directs the output of the command
 - Not required for all commands
- **File Name/Location:** Tells the OS to which file you want the command and options to happen
- Like English sentences, Command Syntax can get very complex

Source: <http://www.linfo.org/cat.html>

Basic Navigation Commands

- `pwd`
 - “Present Working Directory”
 - Prints out your current working directory
- `ls` [FILE]...
 - “List Segments”
 - Optional file/directory paths as an argument
- `cd` [dir]
 - “Change Directory”
 - Optional directory path as an argument
- Absolute paths
 - Starts from the *root directory* (/)
 - `cd /home/cyberpatriot/Music`
- Relative paths
 - Start from the *current directory* (.)
 - `cd ./Music` or just `cd Music`
 - One dot (.) indicates the *current directory*
 - Two dots (..) indicates the *parent directory*

The Basics

Command	Details
who	This will display information about who is logged on, both yourself and others.
whoami	This will display information about the current user i.e. You!
ls	ls is short hand for LiSt. It will display all the visible files within the directory that you are currently in. The files will be listed alphabetically by default but this can be altered with flags.
pwd	This will Print the Working Directory. A fancy way of asking the computer to display information about where you are and what directory you are currently working in.
apropos <i>Keyword</i>	This command followed by a keyword will provide you with information on commands that are related to that keyword.
man <i>Command</i>	This will provide you with the manual for the given command.

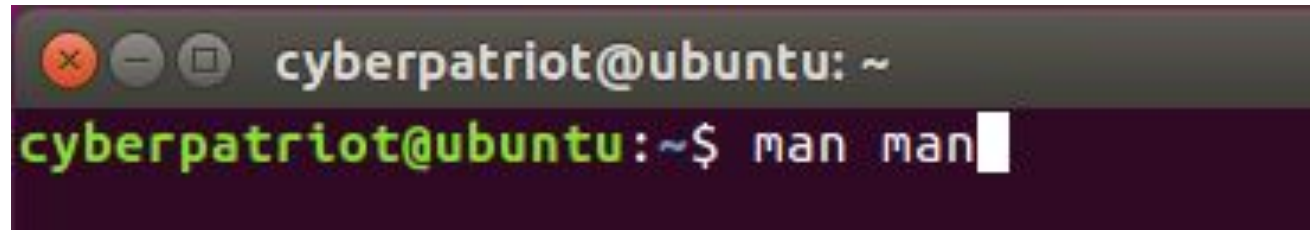
Using these commands you can gather information about the digital world around you, using flags will give you even more control.

We will place the flags behind the ls command for demonstrative purposes

Flag	Details
ls -a	LiSt All – This will list all of the files in the directory including the hidden dot files.
ls -l	LiSt Long – This will list all the files and/or subdirectories with detailed information such as file size, date modified etc.
ls -t	LiSt Time – This will sort the list into a chronological order based on modification time.
ls -tu	This list will list them chronologically based on the last access date.
ls -R	This list will include the content of subdirectories.
ls -S	LiSt Size – This list will be based on the size order.
ls -h	LiSt Human – this will make the list human readable format for example the size would be 1.0MB instead of 1004638
ls -r	LiSt Reversed – This will list the files in a reverse alphabetical order.

Command Manuals and Usage

- `man [section] page`
 - “Manual”
 - Displays the manual for a command
- Type `man man` and press `Enter`
 - Displays the manual for the command “man”
 - Use the arrow keys or `PgUp/PgDn` to scroll up and down

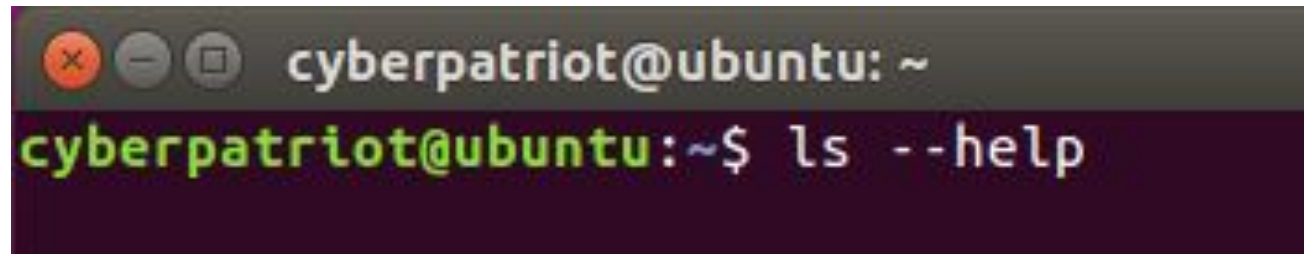
A terminal window with a dark background. The title bar shows window control buttons and the text 'cyberpatriot@ubuntu: ~'. The prompt 'cyberpatriot@ubuntu:~\$' is followed by the command 'man man' and a white cursor at the end.

```
cyberpatriot@ubuntu:~$ man man
```

- Type `q`
 - Exits man

Command Manuals and Usage

- Many commands have a `--help` or `-h` option
- Type `ls --help` and press Enter
 - Displays help for the command `ls`

A screenshot of a terminal window. The title bar shows three window control buttons (red, yellow, green) and the text 'cyberpatriot@ubuntu: ~'. The terminal content shows the prompt 'cyberpatriot@ubuntu:~\$' followed by the command 'ls --help' entered in green text.

```
cyberpatriot@ubuntu: ~  
cyberpatriot@ubuntu:~$ ls --help
```

File Contents and Output Redirection

- `cat` [FILE]...
 - “Concatenate”
 - Concatenate files and prints to standard output
 - Commonly used to print the contents of a single file
- `file` [FILE]...
 - determines the type of a file
- `echo` [STRING]...
 - displays a line of text in the command line
- [`command`] > [FILE]
 - The standard output of any command can be redirected to a file with a “greater than” symbol
 - This will create a new file or overwrite an existing file

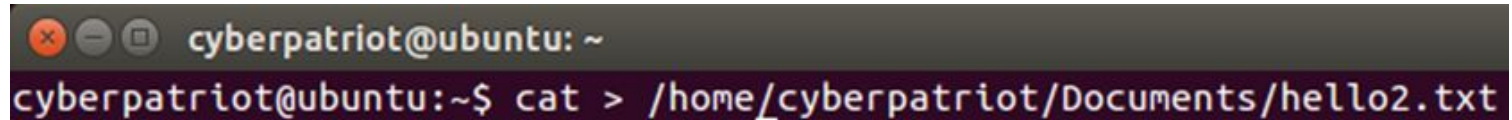
Sample Command

- Note: If the Linux Operating System does not have a CyberPatriot Directory, use another directory

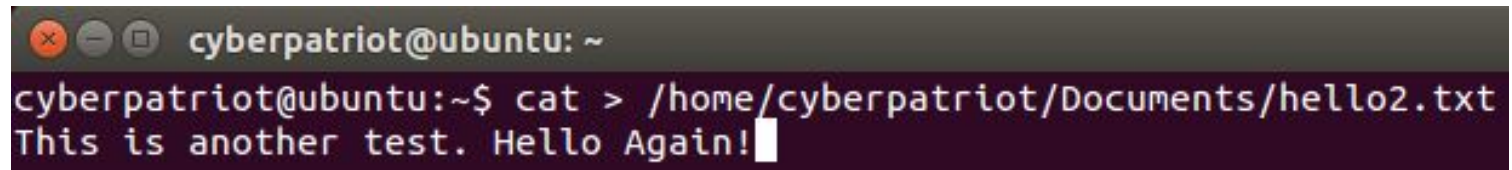
1. In Terminal, type

```
cat -n > /home/cyberpatriot/Documents/hello2.txt
```

**Make sure to capitalize Documents and to put the spaces before -n, >, and /home

A terminal window titled 'cyberpatriot@ubuntu: ~' showing the command 'cat > /home/cyberpatriot/Documents/hello2.txt' being entered at the prompt 'cyberpatriot@ubuntu:~\$'.

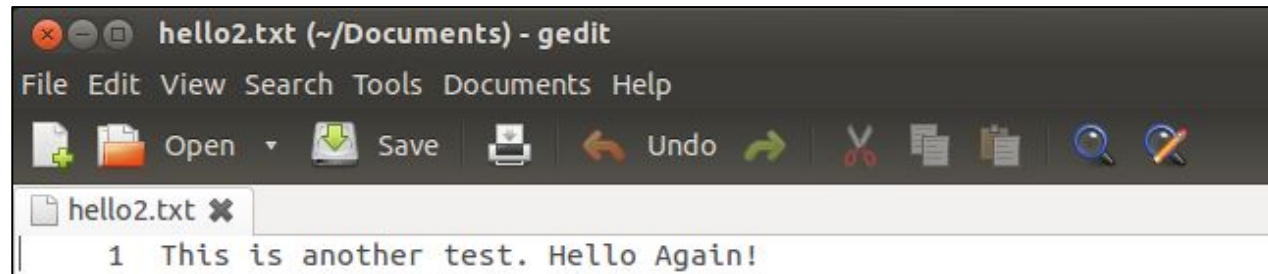
2. Hit **Enter** to execute the command
3. Type **This is another test. Hello Again!**

A terminal window titled 'cyberpatriot@ubuntu: ~' showing the command 'cat > /home/cyberpatriot/Documents/hello2.txt' being entered at the prompt 'cyberpatriot@ubuntu:~\$'. The text 'This is another test. Hello Again!' is being entered on the next line, with a cursor at the end.

2. Hit **Enter** to execute the command
3. Type **Ctrl+D** to close your commands

Sample Command (cont.)

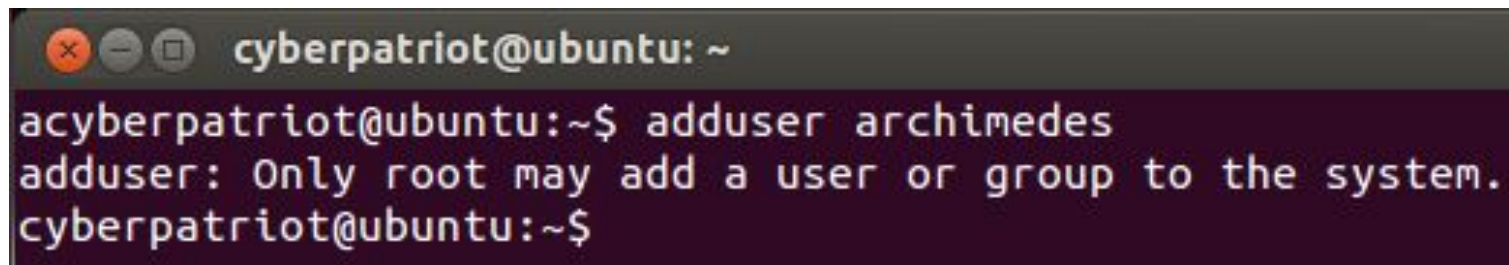
1. Close Terminal and open the Home Folder by clicking the orange folder on the Ubuntu menu bar
2. Navigate to the Documents folder
3. Double-click the **hello2.txt** file



- The commands you just entered created this text document
 - It includes the file name you selected, the text typed, and a “1” at the beginning of the line of text. The next few slides will examine why.

The sudo Command

- Allows an authorized user (one with root permissions) to temporarily elevate their privileges using their own password instead of having to know the password belonging to the built-in root account
- This command must be used to perform administrative tasks (e.g. adding a user account)
 - Example: To add “archimedes” as a user on your system, type `adduser archimedes` and hit `Enter`
 - You will get the error message below because you have not authenticated yourself
 - Note: user names must be lower case



```
cyberpatriot@ubuntu: ~  
acyberpatriot@ubuntu:~$ adduser archimedes  
adduser: Only root may add a user or group to the system.  
cyberpatriot@ubuntu:~$
```

The sudo su Command

- The `sudo su` command is a variation of the `sudo` command
 - It tells the command line that you want to run all of the subsequent commands in your current session as `root`, so that you do not have to enter the `sudo` command and your password each time

Concepts: Files and Processes

In Unix, and by extension, Linux, everything is either a file or a process. Meaning everything can be interfaced via the file system(s).

- ? Files: text, data, documents, traditional files
- ? Directories: directories are special text files that contain a bunch of other files
- ? Devices: all disks, video hardware, audio hardware, processors, memory, USB ports – all hardware can be interfaced via files (usually in /dev)
- ? Processes: all running processes can be “seen” via the file system (in /proc) – each has a unique identifier (PID)



More on Files

- ? Each file (and directory) has a name
- ? The filename can contain letters, numbers and special characters – best to start with name or number
- ? Every file has a unique path to its location
Example: /home/student2/read-write.txt
- ? A filename **MUST** be unique within a directory...though files with the same filename can exist in different directories
- ? Like Linux commands, filenames are case sensitive so a file named “myfile” and “Myfile” can co-exist in the same directory – the names are different.
- ? Filenames can be lengthy

More on Directories

- ? A directory is a special type of file that can hold other files – often referred to as a folder in Windows or MacOS
- ? The “working directory” is the directory with which your shell is currently associated...where you currently are! When you first login, you will normally be in your home directory, /home/username
- ? Use the ‘**pwd**’ command to print working directory
- ? Special directory notations:
 - ? . refers to the current working directory
 - ? .. refers to the parent directory (one level back – the parent directory of /home/username would be /home)

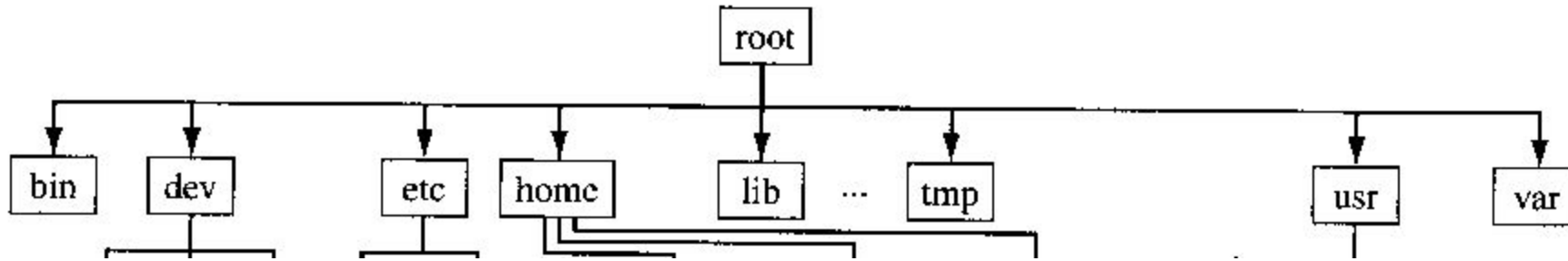
Pathname

? Every file has a unique path to its location...for example:

/home/student2/Projects/docs/final_report.doc

- ? '/home/student2' is the home directory for student2
- ? 'Projects' is a directory in /home/student2 - student2 is the parent directory of Projects
- ? 'docs' is a directory in 'Projects' – docs is a subdirectory or child directory of Projects
- ? 'final_report.doc' is a file in 'docs'

Concepts: The File System



- ? Linux and Unix-like file systems are arranged in a tree structure, all with the same bottom level, called “root” (/).
- ? Unlike Windows there are no drives, drive letters or any separate conceptual “space” for storage hardware.
- ? New hardware will come in the form of a “file system” attached (mounted) to some arbitrary point in the directory structure.

Linux Directory Structure

- ? / root
- ? /bin bare essential commands
- ? /boot OS Kernels
- ? /dev hardware devices
- ? /etc system files, configuration
- ? /home home directories
- ? /lib Libraries needed by the system
- ? /opt 3rd party applications
- ? /proc Running processes
- ? /sbin administrative commands
- ? /tmp temporary space
- ? /usr operating system applications
- ? /var Logs, databases and other variable length stuff

cd and ls commands

- ? The “cd” command is used to change directory location. Without an argument, “cd” takes you to your home directory
- ? The “ls” command is used to list the files in a directory. Like many Linux commands, it can take a number of “flags” as options to change the behavior of the command

```
$ cd /home/$USER
$ ls
$ cd /etc
$ pwd
$ ls
$ ls -l
$ ls -rl
$ cd
$ pwd
```

Exercise #2: “cd” and “ls” commands

- ? Type “cd /data/classes/linux”
- ? Try “ls -l”
- ? Try “ls -a”
- ? Try “ls -la”
- ? How are the above outputs different?
- ? Type “ls -lt”
- ? How is this output ordered?
- ? Type “ls -l /tmp”

The above shows how providing an argument to the ‘ls’ command displays the contents of a directory without first changing to the directory

- ? Type “cd /home/\$USER” to change to your home directory
- ? Now type “cd -”
- ? To what directory did that take you?
- ? Now type “cd ..”
- ? To what directory did that take you?
- ? Type “cd /home/\$USER” to get back to your home directory



Finding your way home!

- ? The “~” is a special character that is short-hand for “/home/username”
- ? The shell variable \$HOME also stores the path of “/home/username”
- ? Several ways to get to your home directory:

```
$ cd ~  
$ cd $HOME  
$ cd /home/username  
$ cd /home/$USER  
$ cd
```



- ? But you can also use the “~” and \$HOME as arguments with other commands:

```
$ ls ~/tmp  
$ ls $HOME/LinuxClass
```

Absolute and Relative paths

- ? The starting “/” in the directory argument explicitly spells out a pathname – specifying an absolute or full path
- ? No leading “/” means you are specifying a path that is relative to the current working directory.
- ? “cd /tmp” is different from “cd tmp”

```
$ cd /home/username

# Absolute path:
$ cd /home/username/tmp

# Relative path:
$ cd tmp

# Using ~

These do the same:
$ cd ~/tmp
$ cd /home/username/tmp
```

Help!

- ? Many commands provide a '--help' option which will display information on the various options and their means. For example:

\$ ls --help

- ? There is also the "man" command, which will provide a manual listing on the use of standard Linux commands such as ls one page at a time

\$ man ls

- ? One can scroll forward or back one line at a time using the up & down arrow keys and scroll forward or back one page at a time by hitting the 'f' or 'b' keys, respectively.
- ? Exit out of a man page by typing 'q'
- ? Try both of the above commands

Users and Groups

- ? Users are associated with a unique user identification (UID) number that the system uses internally
- ? Users can be real people
- ? Users can be system entities
- ? Users can be herded via groups
- ? Groups also are associated with a unique group identification (GID) number by the system
- ? Groups allow multiple users to access/share the same files

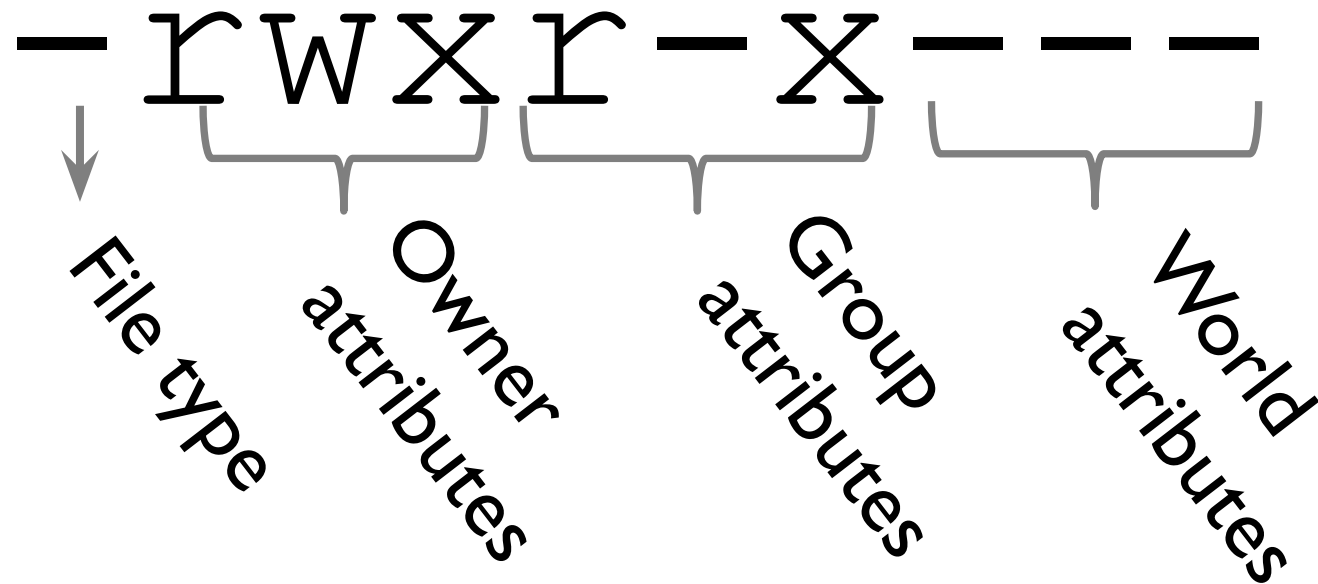


Ownership & Permissions

- ? Linux systems are multi-user environments that allow users to create files, run programs and share data.
- ? Files and directories have two types of ownership – the user and group. A Linux group consists of one or more users.
- ? Files and directories have three types of access permissions:
 - read permission (r)
 - write permission (w)
 - execute permission (x)
- ? Every file and directory has permissions for three levels or entities of permissions:
 - a) user or owner (denoted by u)
 - b) group (one or more users – denoted by g)
 - c) others or world (denoted by o)

Permissions triplets

Each triplet indicates the access permissions for that level – in the example below, the user/owner has read, write & execute permission, other group members only have read and execute permissions and all others have no access permissions.



Permissions (cont)

- Read (file) – view contents
- Read (directory) – see what files and subdirectories it contains
- Write (file) – change contents of , rename, or delete the file
- Write (directory) – add files or subdirectories
- Execute (file) – run the file as a program
- Execute (directory) – cd into the directory

Modifying Permissions

- Use the *chmod* command:

chmod 777 file	rwXrwxrwx
----------------	-----------

chmod 755 file	rwXr-xr-x
----------------	-----------

chmod 644 file	rw-r--r--
----------------	-----------

chmod 600 file	rw-----
----------------	---------

chmod 000 file	-----
----------------	-------

Modifying Permissions (cont)

- Use the *chmod* command with +/-:

- a = all
- u = user
- g = group
- o = other (world)

- Examples:

<code>chmod a-w file</code>	<code>r-xr-xr-x</code>
-----------------------------	------------------------

<code>chmod go-rwx file</code>	<code>r-x-----</code>
--------------------------------	-----------------------

<code>chmod a+w file</code>	<code>rwX-w--w-</code>
-----------------------------	------------------------

Long List Output Explained (a little)

? From left to right:

- Unix permissions
- Hard links
- Owner
- Group ownership
- File size in bytes
- Modification date
- Name of file

```
$ ls -la
drwxrwx--- 104 mark      staff  110592 Aug 17 13:02 .
drwxr-xr-x 2510 root     root   196608 Aug 17 12:58 ..
-rw-r--r--  1 mark      mark    1051 May  8  2016 ad-week
-rwxr--r--  1 mark      staff    239 May 11  2013 alias.pl
-rw-r----- 1 mark      staff   1185 Jun 22  2014 bp.txt
-rwxr-xr-x  1 root      root   27320 Mar 29  2015 getpass.awk
-rw-rw-r--  1 david     staff   20529 Aug  7  2017 httpd.conf
-rwxr--r--  1 root      staff  136236 Sep 10  2017 memcon
drwxr-x---  2 mark      staff   4096 Jun 24  2017 misc
drwx----- 3 mark      staff   4096 Jun 24  2017 test
-rwx----- 1 mark      staff    493 Feb 10  2016 unlock
-rw-r----- 1 mark      staff    38 Oct 20  2017 world.c
-rwxr-x---  1 mark      staff   6703 Jan  8  2017 world.exe
-rwxrwx---  1 mark      staff   2350 May 22  2017 year.pl
lrwxrwxrwx  1 mark      staff    7 Aug 16 15:30 year2 -> year.pl
```

Special Directories:

- . is the current working directory
- .. is the “parent” directory, one level “back”

Permissions described:

File Type:

- “-” regular file
- “d” directory
- “l” symlink
- “b” block device
- “c” character device
- “p” named pipe
- “s” socket

Permissions

- “r” read
- “w” write
- “x” execute

Changing Permissions and Ownership

? Use '**chmod**' to change the file permissions:

```
chmod [ugoa][+/-][rwx] filename
```

where u=user, g=group, o=others or world and a=all three

For example, to provide group read access to a file:

```
$ chmod g+r myfile
```

Or to remove file access to another than the owner or group members (in other words, others):

```
$ chmod o-rwx myfile
```

? The '**chown**' command is used to change file ownership and the '**chgrp**' command can change group ownership of a file. **As a regular user, you can not change the ownership of a file, but you can change the group ownership if you are a member of the group to which you are changing the group ownership**

? You can use the **-R** argument on any of the above to recursively make changes on a directory of files

Exercise #3 pre-exercise

? We'll talk about some of these commands shortly, but you each need to make a copy of files needed for the rest of class.

? First go to your home directory

```
$ cd /home/$USER
```

? Make a 'LinuxClass' directory for yourself and go into that directory:

```
$ mkdir LinuxClass
```

```
$ cd LinuxClass
```

? Copy the files from the exercise file to your directory:

```
$ cp -r /data/classes/linux/* .
```

```
$ ls -l
```

? Create a shell variable to your class directory:

```
$ class=$PWD
```

```
$ cd $class
```

Exercise #3: File Permissions

Read Permissions

The cat command displays contents of a file

```
$ cd /home/$USER/LinuxClass
$ cat read-write.txt
```

Change the read permission

```
$ chmod u-r read-write.txt
$ cat read-write.txt
```

What happened? Now restore the read permission

```
$ chmod u+r read-write.txt
$ cat read-write.txt
```

Execute Permissions

```
$ cd /home/$USER/LinuxClass
```

Run the myhostname file to see the system name

```
$ ./myhostname
```

Remove the execute permission for the user on the myhostname file:

```
$ chmod u-x myhostname
$ ./myhostname
```

What happened?

```
$ chmod u+x myhostname
$ ./myhostname
```

Change permissions on the directory dir-perms:

```
$ chmod u-x dir-perms
$ ls dir-perms
$ ls -l dir-perms
```

What happened and why?

Wildcards

? With many Linux commands, you can use wildcards to match characters

? The '*' can be used to match **zero or more** characters

Examples:

```
$ ls bear*
```

```
bears bears7 bears_chicago
```

```
$ ls *bear*
```

```
bears bears7 bears_chicago polarbears
```

? The '?' can be used to match **EXACTLY one** character

```
$ ls bears?
```

```
bears7
```

Special Keys

Tab Key allows for command auto-completion

Single Quote vs Back Tick

Arrow keys allow you to:

- a) Move horizontally along the command to make changes without deleting and retyping everything
- b) Move vertically through your history of previously run commands



Ctrl-c, ESC & command line editing

- ? If you get into a situation where you can't seem get back to a command line prompt, try hitting the Ctrl-C combination or the Esc key – often one of those will abort whatever you are currently doing.
- ? The Ctrl character is often represented by the ^
- ? Use Ctrl-a to go to the beginning of the line
- ? Use Ctrl-e to go to the end of the line

What is that file?

file

The "file" command tells us what type of file it might be - text, executable, PDF, jpg, gzip, tar, directory, etc.

```
$ file read-write.txt
```

```
$ file world.exe
```

```
$ file examples
```

```
$ file Linux_slides.pdf
```

cat and echo

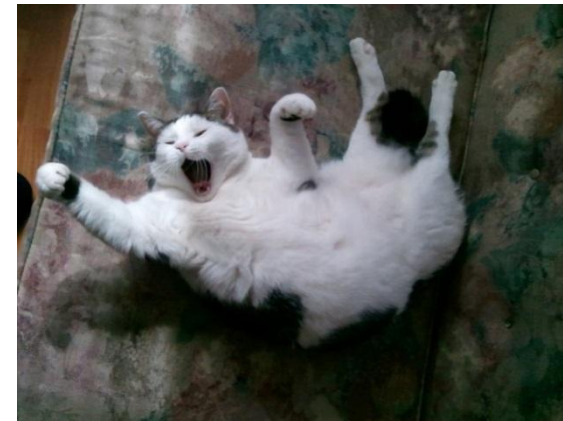
Use cat to display file contents to the terminal:

```
$ cat bears
$ cat bears7
$ cat bears bears7
```

“cat” is short for concatenate. The “cat” command takes one or more files and concatenates their contents to standard output.

echo is used to output arbitrary text to the terminal:

```
$ echo 'Hello World!'
$ echo without single quotes
$ echo 'Hello World!' > MyWorld
```



Output Redirection to Files

Redirect output (>):

```
$ cat bears > Teddybears
$ cat Teddybears
$ cat bears bears7 > bothbears
$ cat bothbears
$ cat bears* > allbears

$ echo 'Hi there!' > greeting
$ cat greeting
```

Append files (>>):

```
$ echo 'Hi yourself!' >> greeting
$ cat Teddybears >> greeting
$ cat greeting
```

Exercise #4: cat and echo

cat a file to view contents

```
$ pwd
$ cd /home/$USER/LinuxClass
$ cat lions
$ cat tigers
$ cat bears
$ cat lions tigers > animals
$ cat animals
$ cat bears >> animals
$ cat animals
```

Using echo

```
$ echo my name is Chris
$ echo "my name is $USER" > myname
$ cat myname
$ echo Hello $USER >> myname
$ cat myname
```

Symbolic links (like windows shortcuts)

للاطلاع فقط

Allows you to reference same file with different name or path – a symbolic link is a another file type.

```
$ ln -s <existing_file> <file_link>
$ cat Capitals
$ ln -s Capitals CapCities
$ ls -l Cap*
$ cat CapCities
```



Example:

```
$ ln -s /home/$USER/LinuxClass/examples/tmp/colors color-pairs
$ ls -la color-pairs
$ cat /home/$USER/LinuxClass/examples/tmp/colors
$ cat color-pairs
```

Creating files/directories

Using touch and mkdir

To create an empty file, use the touch command:

```
$ touch my_data_file
```

You can also create a file using an editor such as pico, nano, vi or emacs:

```
$ nano Music
```

To create a directory:

```
$ mkdir Mydirectory
```

```
$ mkdir 2017
```

```
$ mkdir -p 2018/Jan/stats
```

Deleting files/directories

Using rm and rmdir

To remove a file:

```
$ rm my_data_file
```

```
$ touch myFile
```

```
$ chmod u-rwx myFile
```

```
$ rm myFile
```

```
$ rm -f myFile
```

To remove a directory:

```
$ rmdir Mydirectory
```

```
$ rm -r 2017
```

rmdir only works if the directory is empty!

Dangerous:

```
$ rm -rf *
```

Exercise #5: Creating and deleting files

Creating a file, directory & symbolic link

```
$ cd /home/$USER/LinuxClass
```

```
$ echo 'I love genomic research!' > science
```

Now create a file named science_project and a directory named scienceclass (hint: use touch & mkdir)

```
$ ls -ld science*
```

```
$ ln -s /home/$USER/LinuxClass/examples/tmp/ice_cream ICECREAM
```

```
$ ls -la ICECREAM
```

```
$ cat ICECREAM
```

Deleting a file and directory

```
$ rm science*
```

```
$ ls -ld science*
```

What happened?

```
$ rmdir scienceclass
```

Displaying Portions of a File

“more” or “less”

```
$ more  mascots.txt
$ less  mascots.txt
- each prints out a page of a file at a time
```

“head” or “tail”

```
$ head  mascots.txt
- prints out the first 10 lines by default. Can use the -n argument to change the
  default number of lines

$ tail  -20  mascots.txt
- prints out the last 20 lines
```

Text editors

Good simple editors:

- ? pico (pine composer)
- ? nano (pico clone)

Advanced editors with more features:

- ? “vim” (vi-improved)
- ? “emacs” (Editor MACroS)

nano – a simple editor

\$ nano filename

The essentials:

- ? Just start typing – can use arrow keys to position, backspace or delete key to delete characters to the left
- ? Keystrokes for basic commands at bottom of the screen
- ? ^G – help screen (^C to exit help)
- ? ^O – save the file
- ? ^W – search for a string
- ? ^X – exit nano

\$ nano --help

Exercise #6: Editing a file using nano

```
$ cd /home/$USER/LinuxClass
```

```
$ nano bashrc
```

- 1) Using the RIGHT arrow key, position the cursor at the end of the first line,
 - 2) Use the Backspace key to remove **<HOMEDIRECTORY>** from the first line and then type **\$HOME** after the colon
 - 3) Hit Ctrl-W (to search), type **NANOPATH** and hit Enter – this should place you on the last line
 - 4) Hit Ctrl-E to get to the end of the line
 - 5) Use the Backspace key to remove everything after the '=' sign and type **'/bin/nano'**
 - 6) Use the up & the right arrow keys to get to the @ on the 2nd line
 - 7) Backspace to remove **<USERNAME>** and type **your username**
 - 8) Use the down arrow key to get to the 3rd line
 - 9) Hit Ctrl-K to cut the 3rd line
 - 10) Hit the Up arrow to get to the 1st line & Ctrl-A to get to the start of the line
 - 11) Hit Ctrl-U to paste the text – the 3rd line should now be the first
 - 12) Hit Ctrl-X to exit – type Yes to save the file when prompted and hit Enter when prompted for the name
- ```
$ cat bashrc
```

# mv - moving files/directories

**mv - move one or more files or rename a file (some Linux versions have a 'rename' command, but not all):**

**Syntax: mv source destination**

```
$ touch football
$ touch footballgame
$ ls -l football*
$ mv footballgame footballtee
$ ls -l football*
$ mkdir sports
$ mv sports Sports
$ ls -ld *ports*
$ touch footballtee2
$ mv footballtee* Sports
$ ls -la Sports
```

# cp - copying files/directories

## cp - copy one or more files or directories

**Syntax:** `cp source destination`

```
$ cp football football2
$ cp -p football football3
$ ls -la football*
$ cp -p Sports/footballtee .
$ mkdir -p Sports/fall
$ cp -p football2 Sports/fall
$ cd Sports/fall
$ cp -p football2 ../football4
$ cd ..
$ ls -R Sports
```

**Archival copy:**

```
$ cp -a Sports Sports2017
$ cp -pr Sports Sports2018
```

# Exercise #7: Moving/Copying Files

## Move (mv)

```
$ cd /home/$USER/LinuxClass
$ touch Raspberry
$ mv Raspberry raspberry
$ echo blueberry > blueberry
$ ls -la *berry
```

*Now create a directory path using mkdir with the -p option:*

```
$ mkdir -p Berries/All/B
Use mv to move the blueberry file into
Berries/All/B directory:
$ mv blueberry Berries/All/B
$ touch Berries/All/B/blackberry
$ mv Berries/All/B/blackberry .
$ mv Berries BERRIES
$ ls -Rl Berries
$ ls -Rl BERRIES
```

## Copy (cp)

```
$ cp raspberry strawberry
$ cp -p raspberry cranberry
$ ls -la *berry
```

*How do the four \*berry files differ?*

```
$ mkdir -p BERRIES/Others
$ cp -p *berry BERRIES/Others
$ mv *berry BERRIES
$ ls -la BERRIES
$ cp -pr BERRIES/Others BERRIES/More
$ ls -Rla BERRIES
```

```
$ cp BERRIES NewBerries
```

*What did you see?*

```
$ cp -pr BERRIES NewBerries
```

Archival copy:

```
$ cp -a BERRIES Berries-save
```

# wc - what's in that file?

## “wc” (word count)

```
$ wc mascots.txt
```

```
345 955 7342 mascots.txt
```

Output shows the number of lines, words and characters in the file

Can use argument to only get one of the three values:

```
$ wc -l mascots.txt
```

```
$ wc -w mascots.txt
```

```
$ wc -m mascots.txt
```

```
$ wc -help
```

# grep – pattern matching search of a file

---

## “grep” – global/ regular expression/ print

```
$ grep cat nonsense.txt
$ grep dog nonsense.txt
$ grep -i dog nonsense.txt # case insensitive
$ grep -v dog nonsense.txt # exclude 'dog'
$ grep -A1 cat nonsense.txt # include line after match
$ grep -B1 cat nonsense.txt # include line after match
$ grep oc nonsense.txt
$ grep -c oc nonsense.txt # count of matching lines
$ grep ^oc nonsense.txt # ^ -starts line with oc
$ grep oc$ nonsense.txt # $ - ends line with oc
```

**grep** is a powerful tool. Use it (as well as egrep...extended grep)

```
$ grep --help
$ man grep
```

## Exercise #8: Using grep

? Make sure you are in the class directory:

```
$ cd ~/LinuxClass
$ pwd
```

- ? Using the 'grep' utility with the file **mascots.txt**, determine the following:
- find the lines that have the letters 'cat' (*just lowercase letters*) as part of the mascot name
  - find the lines that have the letters 'cat' **regardless of case** as part of the mascot name
  - find which colleges that **start** with the word "Saint" and redirect the output to a file named **Saints** – how many?

# find – where are my darn files?

find - used to locate files based on various criteria

```
? $ find [path_to_search] [expression/options]
? $ find . -name "*.txt"
? $ find /home/$USER/LinuxClass -iname "capital*"
? $ find /home/$USER/LinuxClass -type f -mmin 40
? $ find /home/$USER/LinuxClass -type f -mmin -40
? $ find /home/$USER/LinuxClass -type f -mmin +40
? $ find /home/$USER/LinuxClass -type f -mtime 1
? $ find /home/$USER/LinuxClass -name "*.bak" -delete
? $ find . -name "*.txt" -exec ls -la {} \;
? $ man find
```

# Exercise #9: Using find

- ? Let's use the find utility to look for files in your home directory
- ? Make sure you are in your class directory:

```
$ cd ~/LinuxClass
$ pwd
```

- ? Using 'find', locate the file named 'colors'
- ? Using 'find', locate the files whose name **contains** the word 'bear' where the match is **case insensitive** (hint: -iname option)
- ? Using 'find', locate the files that were modified LESS than 45 minutes ago (hint: -mmin option)
- ? How many files did you find for each?

# uniq – show or remove duplicate lines

---

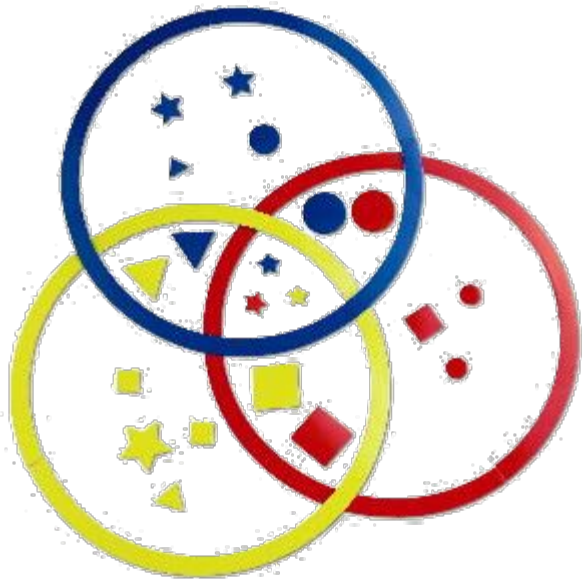
- ? uniq – show either unique or duplicate **consecutive** lines in a file or output. Default behavior is to merge adjacent matching lines into one, but can be used to print just the matching lines or provide a count of matching lines...most effective with the sort command

```
$ uniq bears # will show all unique lines
$ uniq -d bears # show only duplicate lines
$ uniq -c bears # show a count of each unique line
```

# Sorting

## sort command

“sort” can be used to read a file, sort its contents and output to the terminal



```
$ cat grades.txt
$ sort grades.txt
$ sort -r grades.txt
$ sort -k2 grades.txt
$ sort -b -k2 grades.txt
$ sort -bn -k2 grades.txt
$ sort -bnr -k2 grades.txt
$ sort -help
```

# Pipes (redirect to other processes)

Much like you can write output to files, you can write or “pipe” output to other commands using pipes “|”



```
$ cat college1 | sort | uniq
$ cat college2 | sort | uniq
$ cat college1 college2 | sort | uniq -c
```

Write to a file at the end:

```
$ cat college1 college2 | sort | uniq |grep ^B > Colleges
```

# Exercise #10: sort, pipes and redirection

---

- ? `cd /home/$USER/LinuxClass`
- ? Look at the contents of two files, `grocery1` and `grocery2` (use `cat` command)
- ? Combine the two files using the `cat` command and then use the `sort` and `uniq` commands to get a list of sorted, unique items for the grocery list
- ? Now redirect the output to a file named `grocery3`
- ? Use the `wc` command to determine how many unique items are on the list (in the `grocery3` file).
- ? Use `grep` and `wc` to determine how many items in the `grocery3` list start with the letter 'c'

# Exercise #10 continued

---

```
? $ cat grocery1
```

```
? $ cat grocery2
```

```
? $ cat grocery1 grocery2 | sort | uniq
```

```
? $ cat grocery1 grocery2 | sort | uniq | wc -l
```

You should have 32 items

```
? $ cat grocery1 grocery2 | sort | uniq > grocery3
```

```
? $ grep ^c grocery3
```

7 items start with the letter c

# Other useful commands

---

- ? **history** – displays a history of commands which allows for an easy way of running a command again without having to type it out again
- ? **alias** – list aliases or create a new one for another cmd

Example:

```
$ alias hist="history 20"
```

- ? **cut** – print out selected fields

Example:

```
$ cat famousdogs | cut -f1,4 -d:
```

- ? **diff** – find the differences between two files:

```
$ diff numbers1 numbers2
```

# date & cal

---

? date – prints the current date and time

\$ date

Wed Sep 12 15:08:17 EDT 2018

\$ date +“%D %T”

09/12/18 15:08:18

? cal – print the calendar for the current month or entire year

\$ cal 2018

# Input, Output and Error

للاطلاع فقط

- ? Commands can have an input and output
- ? STDIN or 'standard input' is input from the keyboard though we can have redirected input from a file
- ? STDOUT or 'standard output' is output going to the screen. We've already seen where we can 'redirect' the output of a command to a file or pipe it as the input to another command
- ? Commands may also produce errors such as 'Permission denied'
- ? STDERR or 'standard error' is error output that goes to the screen by default

# Input, Output and Error (cont)

للاطلاع فقط

? STDIN, STDOUT and STDERR have handles or numbers associated with each:

|        | Handle |                 |
|--------|--------|-----------------|
| STDIN  | 0      | Standard input  |
| STDOUT | 1      | Standard output |
| STDERR | 2      | Standard error  |

Let's change permissions on dogs2 to be unreadable:

```
$ cd $HOME/LinuxClass
```

```
$ chmod ugo-r dogs2
```

? Can redirect the STDERR to a file:

```
$ grep dog dogs* 2> errors
```

```
$ grep dog dogs* 2> /dev/null
```

? Can redirect BOTH the STDOUT and STDERR to a same file:

```
$ grep dog dogs* > out_plus_errs 2>&1
```

or to different files:

```
$ grep dog dogs* > OUTPUT 2> ERRORS
```

# Putting it all together

Read in from a file with input redirection, do some stuff and output to another file:

```
$ sort < Colleges.txt | grep -i ^C > C-colleges
```



- ? Program first, then arguments, then any file I/O
- ? Most programs will read from standard input (stdin) if no file is specified in arguments

# awk – text manipulation

- ? In awk, lines are broken down into fields that are represented by variables \$1, \$2, \$3, etc
- ? 'print \$1' will print the first field
- ? Let's look at an example:

```
$ cat hare_tortoise
```

The hare beat the tortoise handily.

- ? We can change the ordering of words using awk:

```
$ cat hare_tortoise | awk '{print $1,$5,$3,$4,$2,$6}'
```

The tortoise beat the hare handily.

- ? You do NOT have to use all of the fields – can pick and choose as needed

# sed – stream editor for pattern matching and modification

? In sed, one can do text pattern matching and modification

```
$ cat hare_tortoise
```

The hare beat my tortoise handily.

? If we want to change the word ‘beat’ with the word ‘defeated’:

```
$ cat hare_tortoise | sed 's/beat/defeated/g'
```

The hare defeated my tortoise handily.

? The trailing ‘/g’ at the end of that command indicates that the change is to be done globally...without it, only the first occurrence of the word in the file will be changed.

? We can use both awk & sed on the same command line:

```
$ cat hare_tortoise | awk '{print $1,$5,$3,$4,$2,$6}' |
sed 's/beat/defeated/g'
```

The tortoise defeated my hare handily.

# tr – allows character substitution or translation

? With tr, characters can be used to be translated – perhaps to change the case of letters:

```
$ echo 'Let's Go Caps!!!' | tr "a-z" "A-Z"
LET'S GO CAPS!!!
```

□ Or to replace a new line character (\n) with a space or comma:

```
$ cat numbers2 | tr "\n" " "
1 2 3 4 5 6 7 8
$ cat numbers2 | tr "\n" ", "
1,2,3,4,5,6,7,8,
```

# More Linux Command Basics...Quotes

- ? Linux treats single, double and back quotes in commands differently
- ? Contents of a set of single quotes are treated as a string:

```
$ echo '$USER'
$USER
```

- Contents of a set of double quotes will have any included variables replaced:

```
$ echo "The home directory of $USER is $HOME"
The home directory of username is /home/username
```

- Contents of a set of back quotes or back ticks (on the upper left of the keyboard) are treated as a command and the output can be assigned to a variable:

```
$ NOW=`date`; echo $NOW
Mon Jul 30 15:08:56 EDT 2018
```

# Your PATH

---

## Execution path

In BASH, execution of a program happens when you enter the program name. Your PATH variable keeps you from having to enter the full path to the program

```
$ echo $PATH

$ which date
$ which whoami
$ which perl
```

## Modifying your PATH

```
$ echo $PATH
$ PATH=$PATH:/data/$USER
$ echo $PATH
```

To path changes permanent, need to modify the .bashrc file in your home directory.

# Shell Variables

## Variable assignment

### Arbitrary assignment

```
$ MYWORLD="Hello World"
$ echo $MYWORLD
$ MY_PI=3.14
$ echo $MY_PI
```

### With program output

```
$ RIGHTNOW=`date`
$ echo $RIGHTNOW
Mon Jul 30 15:08:56 EDT 2018
```

## From a file

```
$ FILE=`cat nonsense.txt`
$ echo $FILE

$ echo $FILE | tr " " "\n" \
|sort | uniq
```

# Shell Variables

## Show all currently assigned variables

```
$ printenv
HOSTNAME=biowulf.nih.gov
TERM=xterm
SHELL=/bin/bash
HISTSIZE=500
SSH_CLIENT=165.112.93.227 49886 22
OLDPWD=/home/mark
HISTFILESIZE=500
USER=mark
...
```

## Useful predefined and important variables

|              |                   |
|--------------|-------------------|
| ? \$HOSTNAME | System hostname   |
| ? \$USER     | Your Username     |
| ? \$SHELL    | Your shell        |
| ? \$HOME     | Home directory    |
| ? \$PWD      | Current directory |
| ? \$PATH     | Command paths     |

# Loops

“For” loops allow for iteration based on flow control criteria

```
$ for n in 1 2 3 4 5 6 7
> do
> echo The value of n is $n
> done
```

```
$ for n in {1..7}
> do
> echo The value of n is $n
> done
```

```
$ for n in `cat Capitals`
> do
> echo $n|tr "a-z" "A-Z"
> done
```

Perform math using the seq command:

```
$ seq 1 10
```

```
$ t=0
$ for n in `seq 1 10`; do
> t=`expr $t + $n`
> echo $t
> done
$ echo $t
55
```

```
$ t=1
$ for n in `seq 1 10`; do
> t=`expr $t \* $n`
> done
```

# Loops, part 2

Let's combine contents of many named files into one

```
$ cd /home/$USER/LinuxClass/loops
$ cat nih1
.
.
$ cat nih9
```

We can combine the contents of the 9 nih named files into one using a loop:

```
$ for i in `seq 1 9`
> do cat nih$i >> NIH-ALL
> done
$ cat NIH-ALL
```

# Logic tests

---

## Equality: if, then, else, fi

```
$ TRUE="good"
$ if ["$TRUE" = "good"]
> then
> echo "it's true"
> fi

$ if ["$TRUE" = "no good"]
> then
> echo "true statement"
> else
> echo "false statement"
> fi
```

## Existence

```
$ if [-e $HOME/.bashrc]
> then
> echo true
> fi

$ if [-d $HOME/LinuxClass]
> then
> echo true
> fi
```

# Processes

---

## Show processes

Show your processes:

```
$ ps
```

```
$ ps -f
```

sleep is a delay or pause for specified number of seconds

```
$ sleep 5
```

```
$ sleep 25 &
```

```
$ ps -f
```

```
$ ps -f --forest
```

Show all processes:

```
$ ps -e
```

```
$ ps -ef --forest
```

```
$ man ps
```

# More on Processes

## Background and Foreground processes

- ? A command/job can be run in the background by adding '&' to end of the command:

```
$ sleep 50 &
[1]+ Done sleep 50
```

- ? **ctrl-z** suspends a running job/process
- ? **bg** allows you to resume a suspended job in the background and returns you to the command prompt
- ? **fg** allows you to resume a suspended job in the foreground until it completes
- ? **ctrl-c** interrupts or kills the currently running process
- ? **Warning:** Backgrounded processes will die when you log out of your session unless you use something like nohup or screen.

# More on Processes

## Detach and Reattach processes

Ctrl-z suspends an active job

```
$ sleep 300
```

```
[ctrl-z] (process is suspended)
```

```
$ bg
```

```
$ ps -f
```

```
$ fg
```

## Killing a process

```
$ sleep 300
```

```
[ctrl-Z]
```

```
$ bg
```

```
$ ps
```

| PID  | TTY   | TIME     | CMD   |
|------|-------|----------|-------|
| 6686 | pts/0 | 00:00:03 | bash  |
| 8298 | pts/0 | 00:00:00 | sleep |
| 8299 | pts/0 | 00:00:00 | ps    |

(find the PID of the process you want to kill)

```
$ kill 8298
```

# Processes: kill them

“kill” only requests that the program exit. Use a signal 9 to force it to exit

```
$ sleep 300
[ctrl-Z]
$ ps
 PID TTY TIME CMD
 6686 pts/0 00:00:03 bash
 8298 pts/0 00:00:00 sleep
 8299 pts/0 00:00:00 ps
(find the PID of the process you want to kill)
$ kill -9 8298
```

- ? The kill command is slightly misnamed, what it actually does is send a *signal* to a process
- ? Most signals are interpreted by the application being signaled and thus behavior is consistent only by convention
- ? Using signal 9 is dangerous if used indiscriminately

# Exercise: Using 'kill' on a process

- ? First start a 'sleep' process that will run in the background for 300 seconds:  
**\$ sleep 300**
- ? Type '**Ctrl-z**' (the Ctrl and z keys together) to suspend the 'sleep' process
- ? Type '**bg**' to unsuspend the 'sleep' process and have it run in the background.
- ? Check that the process is running by using the '**ps**' command and note the pid, process identification number
- ? Using the '**kill**' command with the pid of the sleep process from the previous step, terminate the sleep process
- ? How can you check that the sleep process is gone?

# uptime

---

? **uptime** shows a summary of the system status

```
$ uptime
14:39:46 up 14 days, 7:00, 305 users, load average: 39.18, 40.68, 38.68
```

? Numbers shown from left to right:

- Current time
- Amount of the time the system has been up
- Number of users currently logged on
- The average system load for the past 1, 5 and 15 minutes
- The load is the usage of the system's CPUs – a load of 1 corresponds to a full load of 1 CPU

# Who is doing what...using top

## By default, top will produce continuous output about running processes

```
$ top
top - 16:19:54 up 28 days, 9:07, 255 users, load average: 32.18, 32.79, 33.22
Tasks: 4749 total, 8 running, 4733 sleeping, 7 stopped, 1 zombie
Cpu(s): 9.6%us, 5.8%sy, 6.0%ni, 78.2%id, 0.2%wa, 0.0%hi, 0.2%si, 0.0%st
Mem: 1058656848k total, 955041356k used, 103615492k free, 79064k buffers
Swap: 67108856k total, 547376k used, 66561480k free, 89619996k cached
```

| PID    | USER      | PR | NI | VIRT  | RES  | SHR  | S | %CPU | %MEM | TIME+     | COMMAND |
|--------|-----------|----|----|-------|------|------|---|------|------|-----------|---------|
| 120202 | johanesb  | 39 | 19 | 235m  | 180m | 1432 | R | 96.4 | 0.0  | 170:21.86 | merlin  |
| 252158 | liqingli  | 39 | 19 | 58496 | 26m  | 756  | S | 95.0 | 0.0  | 17141:15  | moe     |
| 170176 | bozser    | 33 | 13 | 407m  | 117m | 2588 | S | 60.1 | 0.0  | 62:30.33  | ascp    |
| 218983 | jruessler | 20 | 0  | 18532 | 4704 | 872  | R | 22.3 | 0.0  | 0:00.38   | top     |
| 127988 | elliottm  | 39 | 19 | 223m  | 3544 | 1064 | S | 16.8 | 0.0  | 782:02.42 | sshd    |
| 198816 | wenxiao   | 20 | 0  | 4280  | 792  | 416  | D | 14.0 | 0.0  | 24:50.19  | gzip    |

## Hit 'q' to quit out of top



# Looking at file system (disk) Space

**To see local file system space:**

**\$ df -l**

| Filesystem                | 1K-blocks | Used     | Available | Use% | Mounted on |
|---------------------------|-----------|----------|-----------|------|------------|
| /dev/mapper/vg_helix-root | 51403396  | 8502228  | 40289968  | 18%  | /          |
| tmpfs                     | 529355640 | 2216     | 529353424 | 1%   | /dev/shm   |
| /dev/sda2                 | 495844    | 180879   | 289365    | 39%  | /boot      |
| /dev/sda1                 | 204580    | 33228    | 171352    | 17%  | /boot/efi  |
| /dev/mapper/vg_helix-tmp  | 51606140  | 973788   | 48010912  | 2%   | /tmp       |
| /dev/mapper/vg_helix-var  | 32253856  | 19349996 | 11265460  | 64%  | /var       |

**To see numbers in human readable format:**

**\$ df -lh**

| Filesystem                   | Size | Used | Avail | Use% | Mounted on |
|------------------------------|------|------|-------|------|------------|
| /dev/mapper/vg_helix-lv_root | 50G  | 8.9G | 38G   | 19%  | /          |
| tmpfs                        | 505G | 5.6M | 505G  | 1%   | /dev/shm   |
| /dev/sda2                    | 485M | 142M | 318M  | 31%  | /boot      |
| /dev/sda1                    | 200M | 256K | 200M  | 1%   | /boot/efi  |
| /dev/mapper/vg_helix-lv_tmp  | 50G  | 613M | 47G   | 2%   | /tmp       |

**All filesystems, including network file systems:**

**\$ df -h**

# Directory size

---

## Estimate file space use (du)

```
$ cd /home/$USER
```

Estimate a file size:

```
$ du LinuxClass/Linux_slides.pdf
```

Summary:

```
$ du -s LinuxClass
```

Summary in human-readable format:

```
$ du -sh LinuxClass
```

Default behavior:

```
$ du
```

# Checking Quotas on Helix/Biowulf

## “checkquota”

- ? The **checkquota** command will query all network storage devices to find the applicable quota(s) for your user
- ? This command is specific to Helix and Biowulf and is not available to Linux in general since it relies on information that is site-specific to this infrastructure.

```
$ checkquota
```

| Mount  | Used    | Quota    | Percent | Files  |
|--------|---------|----------|---------|--------|
| /data: | 94.2 GB | 200.0 GB | 47.12%  | 70424  |
| /home: | 5.2 GB  | 16.0 GB  | 32.50%  | 133607 |

? The **tar** command allow users to compress and archive files – does not remove the original files by default

## **Syntax for tar to create an archive:**

```
$ tar -czvf <output file> <files to be archived>
```

where c is to create

z is to compress using gzip

v is verbose output (lists files as they are tar'd)

f indicates that the next argument is the output filename

## **Syntax to extract files from an archive:**

```
$ tar -xzvf <filename>
```

## **Syntax to list the files in a tar file:**

```
$ tar -tzvf tarfile.tar.gz
```

? The **gzip** command is similar, but the original file is removed unless you specify to keep it with the '-c' option.

? Example:

```
$ gzip files.gz
```

## Cron: run a job whenever you want

```
? crontab -l
? crontab -e
--
15 3 * * * ~/script.sh >> ~/output 2> ~/error.log-
```

Runs "script.sh" at 3:15AM every day of every week of every month of every year.

```
* * * * *
? First number is the minute at which to run (0-59)
? Second is the hour (0-23)
? Third is the day of the month (1-31)
? Fourth is the month (1-12)
? Fifth is the day of the week (0-6), 0 is Sunday
```

# Resources

---

? **Linux Documentation Project:** <http://www.tldp.org>

? **Introduction to Linux - A Hands on Guide:**

<http://www.tldp.org/LDP/intro-linux/html/index.html>

? **Bash Guide for Beginners:**

<https://www.tldp.org/LDP/Bash-Beginners-Guide/html/>

? **Linux Tutorial at the Texas Advanced Computing Center (TACC):**

<https://portal.tacc.utexas.edu/-/linux-unix-basics-for-hpc>

? **NIH HPC Web Site:** <https://hpc.nih.gov/training>



**part 2**  
**Tutorial of Linux shell script**

What is a shell script?

A shell script is file containing a series of commands.

The shell reads this file and executes the commands sequentially as if they were inputted manually.

## Different types of shells

Shell scripts are interpreted by another programs

These programs establish differences as to how write the scripts to be interpreted

Most commonly known shells are:

- sh**: One of the first ones. Developed for UNIX at AT&T

- bash**: the default shell for many Linux distributions. It has many improvements over *sh*.

- csh** and **tcsh**: short for C-shell. C language style and approach. Not as popular as *bash*.

## How to write and execute shell scripts – Permissions

Shell scripts need execution permission to be able to run:

**`$ chmod u+x test.sh`**

```
[hcen001@ivan-b Tutorial2]$
[hcen001@ivan-b Tutorial2]$ ls
total 15K
drwxrwx--- 2 hcen001 user 8 Jan 29 11:22 .
drwxrwx--- 3 hcen001 user 3 Jan 27 16:17 ..
-rwxrw---- 1 hcen001 user 231 Jan 28 19:11 fileinput.sh
-rw-rw---- 1 hcen001 user 158 Jan 28 19:12 filelist.txt
-rwxrw---- 1 hcen001 user 100 Jan 28 18:54 input.sh
-rwxrw---- 1 hcen001 user 97 Jan 28 18:31 loop.sh
-rwxrw---- 1 hcen001 user 35 Jan 29 11:21 test.sh
-rwxrwx--x 1 hcen001 user 61 Jan 28 21:38 variable.sh
[hcen001@ivan-b Tutorial2]$
[hcen001@ivan-b Tutorial2]$ chmod u+x test.sh
```

Permission granted for the user

## How to write and execute shell scripts – Execute scripts

After permissions are granted, scripts can be executed

**\$ ./test.sh**

```
[hcen001@ivan-b Tutorial2]$
[hcen001@ivan-b Tutorial2]$./test.sh
This is a test
[hcen001@ivan-b Tutorial2]$
```

## Basic Shell Programming

A script is a file that contains shell commands

data structure: variables

control structure: sequence, decision, loop

Shebang line for bash shell script:

```
#!/bin/bash
```

```
#!/bin/sh
```

to run:

make executable: `% chmod +x script`

invoke via: `% ./script`

## Bash shell programming

- Input
  - prompting user
  - command line arguments
- Decision:
  - if-then-else
  - case
- Repetition
  - do-while, repeat-until
  - for
  - select

## User input

shell allows to prompt for user input

### Syntax:

```
read varname [more vars]
```

or

```
read -p "prompt" varname [more vars]
```

words entered by user are assigned to

**varname** and “**more vars**”

last variable gets rest of input line

User input example

```
#!/bin/sh
```

```
read -p "enter your name: " first last
```

```
echo "First name: $first"
```

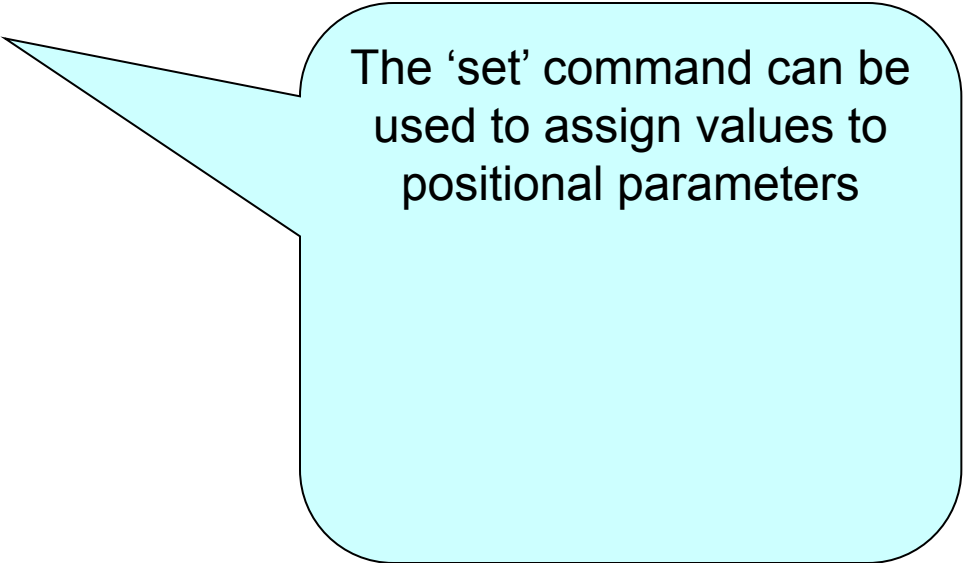
```
echo "Last name: $last"
```

## Special shell variables

| Parameter | Meaning                                              |
|-----------|------------------------------------------------------|
| \$0       | Name of the current shell script                     |
| \$1-\$9   | Positional parameters 1 through 9                    |
| \$#       | The number of positional parameters                  |
| \$*       | All positional parameters, “\$*” is one string       |
| \$@       | All positional parameters, “\$@” is a set of strings |
| \$?       | Return status of most recently executed command      |
| \$\$      | Process id of current process                        |

## Examples: Command Line Arguments

```
% set tim bill ann fred
 $1 $2 $3 $4
% echo $*
tim bill ann fred
% echo $#
4
% echo $1
tim
% echo $3 $4
ann fred
```



The 'set' command can be used to assign values to positional parameters

## **bash control structures**

- if-then-else
- case
- loops
  - for
  - while
  - until
  - select

## if statement

```
if command
then
 statements
fi
```

statements are executed only if **command** succeeds, i.e. has return status “0”

## test command

### Syntax:

**test expression**

**[ expression ]**

evaluates 'expression' and returns true or false

**test** exits with the status determined by EXPRESSION. Placing the EXPRESSION between square brackets ([ and ]) is the same as testing the EXPRESSION with **test**

### Example:

```
if test -w "$1"
then
 echo "file $1 is write-able"
fi
```

The simple if statement

```
if [condition]; then
 statements
fi
```

executes the statements only if **condition** is true

The if-then-else statement

```
if [condition]; then
 statements-1
else
 statements-2
fi
```

executes statements-1 if condition is true

executes statements-2 if condition is false

The if...statement

```
if [condition]; then
 statements
elif [condition]; then
 statement
else
 statements
fi
```

The word **elif** stands for “else if”

It is part of the if statement and cannot be used by itself

## Relational Operators

| Meaning                            | Numeric | String      |
|------------------------------------|---------|-------------|
| Greater than                       | -gt     |             |
| Greater than or equal              | -ge     |             |
| Less than                          | -lt     |             |
| Less than or equal                 | -le     |             |
| Equal                              | -eg     | = or ==     |
| Not equal                          | -ne     | !=          |
| str1 is less than str2             |         | str1 < str2 |
| str1 is greater str2               |         | str1 > str2 |
| String length is greater than zero |         | -n str      |
| String length is zero              |         | -z str      |

## Example: Using the ! Operator

```
#!/bin/bash
```

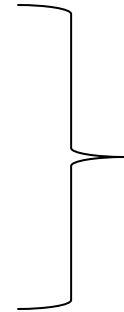
```
read -p "Enter years of work: " Years
if [! "$Years" -lt 20]; then
 echo "You can retire now."
else
 echo "You need 20+ years to retire"
fi
```

## Compound logical expressions

! not

&& and

|| or



and, or

must be enclosed within

[[ ]]

## Single and double square brackets ([,[[)

```
$ [a < b]
-bash: b: No such file or directory
$ [[a < b]]
```

The first example returns an error because bash tries to redirect the file b to the command [ a ]. The second example actually does what you expect it to. The character < no longer has its special meaning of File Redirection operator.

Example: Using the **&&** Operator

```
#!/bin/bash
```

```
Bonus=500
```

```
read -p "Enter Status: " Status
```

```
read -p "Enter Shift: " Shift
```

```
if [["$Status" = "H" && "$Shift" = 3]]
```

```
then
```

```
 echo "shift $Shift gets \$$Bonus bonus"
```

```
else
```

```
 echo "only hourly workers in"
```

```
 echo "shift 3 get a bonus"
```

```
fi
```

Example: Using the **||** Operator

```
#!/bin/bash

read -p "Enter calls handled:" CHandle
read -p "Enter calls closed: " CClose
if [["$CHandle" -gt 150 || "$CClose" -gt 50]]
then
 echo "You are entitled to a bonus"
else
 echo "You get a bonus if the calls"
 echo "handled exceeds 150 or"
 echo "calls closed exceeds 50"
fi
```

## File Testing

### Meaning

|         |                                     |
|---------|-------------------------------------|
| -d file | True if 'file' is a directory       |
| -f file | True if 'file' is an ord. file      |
| -r file | True if 'file' is readable          |
| -w file | True if 'file' is writable          |
| -x file | True if 'file' is executable        |
| -s file | True if length of 'file' is nonzero |

## Example: File Testing

```
#!/bin/bash
echo "Enter a filename: "
read filename
if [! -r "$filename"]
then
 echo "File is not read-able"
 exit 1
fi
```

## Example: File Testing

```
#!/bin/bash

if [$# -lt 1]; then
 echo "Usage: filetest filename"
 exit 1
fi

if [[! -f "$1" || ! -r "$1" || ! -w "$1"]]
then
 echo "File $1 is not accessible"
 exit 1
fi
```

## Example: if... Statement

# The following THREE *if*-conditions produce the same result

\* DOUBLE SQUARE BRACKETS

```
read -p "Do you want to continue?" reply
if [[$reply = "y"]]; then
 echo "You entered " $reply
fi
```

\* SINGLE SQUARE BRACKETS

```
read -p "Do you want to continue?" reply
if [$reply = "y"]; then
 echo "You entered " $reply
fi
```

\* "TEST" COMMAND

```
read -p "Do you want to continue?" reply
if test $reply = "y"; then
 echo "You entered " $reply
fi
```

Example: if..elif... Statement

```
#!/bin/bash
```

```
read -p "Enter Income Amount: " Income
```

```
read -p "Enter Expenses Amount: " Expense
```

```
let Net=$Income-$Expense
```

```
if ["$Net" -eq "0"]; then
```

```
 echo "Income and Expenses are equal - breakeven."
```

```
elif ["$Net" -gt "0"]; then
```

```
 echo "Profit of: " $Net
```

```
else
```

```
 echo "Loss of: " $Net
```

```
fi
```

## The case Statement

use the case statement for a decision that is based on multiple choices

### Syntax:

```
case word in
 pattern1) command-list1
 ;;
 pattern2) command-list2
 ;;
 patternN) command-listN
 ;;
esac
```

case pattern

checked against word for match

may also contain:

**\***

**?**

**[ ... ]**

**[ :class: ]**

multiple patterns can be listed via:

|

## Example 1: The case Statement

```
#!/bin/bash
echo "Enter Y to see all files including hidden files"
echo "Enter N to see all non-hidden files"
echo "Enter q to quit"

read -p "Enter your choice: " reply

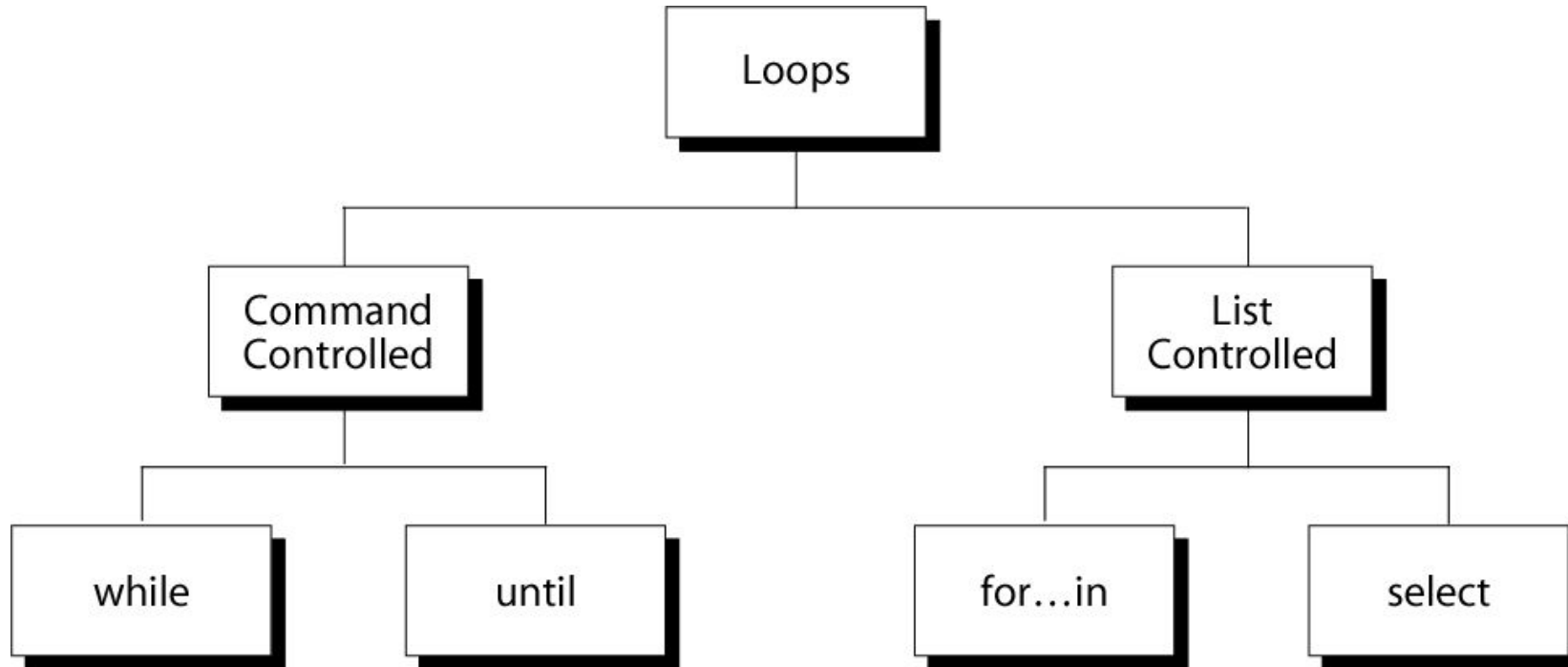
case $reply in
 Y|YES) echo "Displaying all (really...) files"
 ls -a ;;
 N|NO) echo "Display all non-hidden files..."
 ls ;;
 Q) exit 0 ;;

 *) echo "Invalid choice!"; exit 1 ;;
esac
```

## Example 2: The case Statement

```
#!/bin/bash
ChildRate=3
AdultRate=10
SeniorRate=7
read -p "Enter your age: " age
case $age in
 [1-9]||[1][0-2]) # child, if age 12 and younger
 echo "your rate is" '$' "$ChildRate.00" ;;
 # adult, if age is between 13 and 59 inclusive
 [1][3-9]||[2-5][0-9])
 echo "your rate is" '$' "$AdultRate.00" ;;
 [6-9][0-9]) # senior, if age is 60+
 echo "your rate is" '$' "$SeniorRate.00" ;;
esac
```

# Repetition Constructs



## The while Loop

Purpose:

To execute commands in “command-list” as long as “expression” evaluates to true

Syntax:

```
while [expression]
do
 command-list
done
```

Example: Using the while Loop

```
#!/bin/bash
COUNTER=0
while [$COUNTER -lt 10]
do
 echo The counter is $COUNTER
 let COUNTER=$COUNTER+1
done
```

Example: Using the while Loop

```
#!/bin/bash
```

```
Cont="Y"
```

```
while [$Cont = "Y"]; do
```

```
 ps -A
```

```
 read -p "want to continue? (Y/N)" reply
```

```
 Cont=`echo $reply | tr [:lower:] [:upper:]`
```

```
done
```

```
echo "done"
```

## Example: Using the while Loop

```
#!/bin/bash
copies files from home- into the webserver- directory
A new directory is created every hour

PICSDIR=/home/carol/pics
WEBDIR=/var/www/carol/webcam
while true; do
 DATE=`date +%Y%m%d`
 HOUR=`date +%H`
 mkdir $WEBDIR/"$DATE"
 while [$HOUR -ne "00"]; do
 DESTDIR=$WEBDIR/"$DATE"/"$HOUR"
 mkdir "$DESTDIR"
 mv $PICSDIR/*.jpg "$DESTDIR"/
 sleep 3600
 HOUR=`date +%H`
 done
done
```

## The until Loop

Purpose:

To execute commands in “command-list” as long as “expression” evaluates to false

Syntax:

```
until [expression]
do
 command-list
done
```

Example: Using the until Loop

```
#!/bin/bash
```

```
COUNTER=20
```

```
until [$COUNTER -lt 10]
```

```
do
```

```
 echo $COUNTER
```

```
 let COUNTER-=1
```

```
done
```

Example: Using the until Loop

```
#!/bin/bash
```

```
Stop="N"
```

```
until [$Stop = "Y"]; do
```

```
 ps -A
```

```
 read -p "want to stop? (Y/N)" reply
```

```
 Stop=`echo $reply | tr [:lower:] [:upper:]`
```

```
done
```

```
echo "done"
```

## The for Loop

Purpose:

To execute commands as many times as the number of words in the “argument-list”

Syntax:

```
for variable in argument-list
do
 commands
done
```

## Example 1: The for Loop

```
#!/bin/bash
```

```
for i in 7 9 2 3 4 5
do
 echo $i
done
```

## Example 2: Using the for Loop

```
#!/bin/bash
compute the average weekly temperature

for num in 1 2 3 4 5 6 7
do
 read -p "Enter temp for day $num: " Temp
 let TempTotal=TempTotal+Temp
done

let AvgTemp=TempTotal/7
echo "Average temperature: " $AvgTemp
```

## looping over arguments

simplest form will iterate over all command line arguments:

```
#!/bin/bash
for parm
do
 echo $parm
done
```

## Select command

Constructs simple menu from word list

Allows user to enter a number instead of a word

User enters sequence number corresponding to the word

### Syntax:

```
select WORD in LIST
do
 RESPECTIVE-COMMANDS
done
```

Loops until end of input, i.e. ^d (or ^c)

Select example

```
#!/bin/bash
select var in alpha beta gamma
do
 echo $var
done
```

Prints:

```
1) alpha
2) beta
3) gamma
#? 2
beta
#? 4
#? 1
alpha
```

## Select detail

PS3 is select sub-prompt

\$REPLY is user input (the number)

```
#!/bin/bash
```

```
PS3="select entry or ^D: "
```

```
select var in alpha beta
```

```
do
```

```
 echo "$REPLY = $var"
```

```
done
```

```
Output:
select ...
1) alpha
2) beta
? 2
2 = beta
? 1
1 = alpha
```

Select example

```
#!/bin/bash
echo "script to make files private"
echo "Select file to protect:"

select FILENAME in *
do
 echo "You picked $FILENAME ($REPLY) "
 chmod go-rwx "$FILENAME"
 echo "it is now private"
done
```

## break and continue

Interrupt for, while or until loop

The break statement

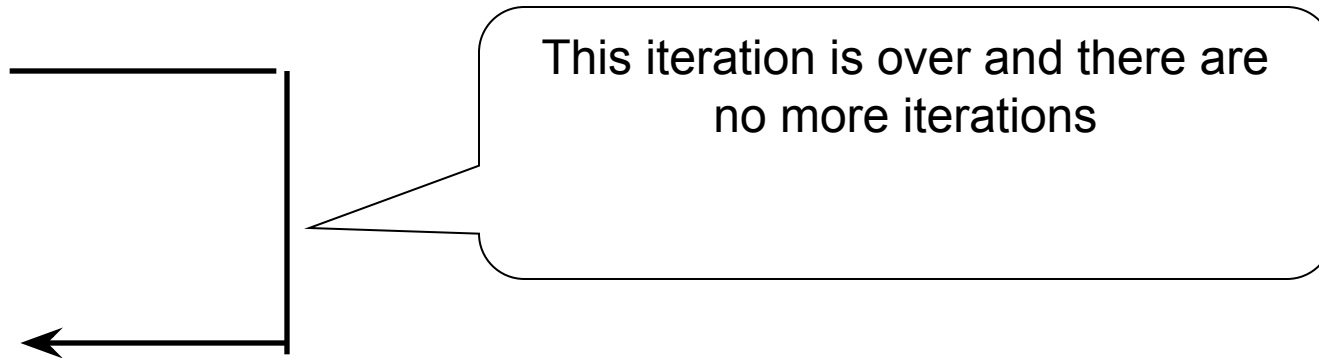
- transfer control to the statement AFTER the done statement
- terminate execution of the loop

The continue statement

- transfer control to the statement TO the done statement
- skip the test statements for the current iteration
- continues execution of the loop

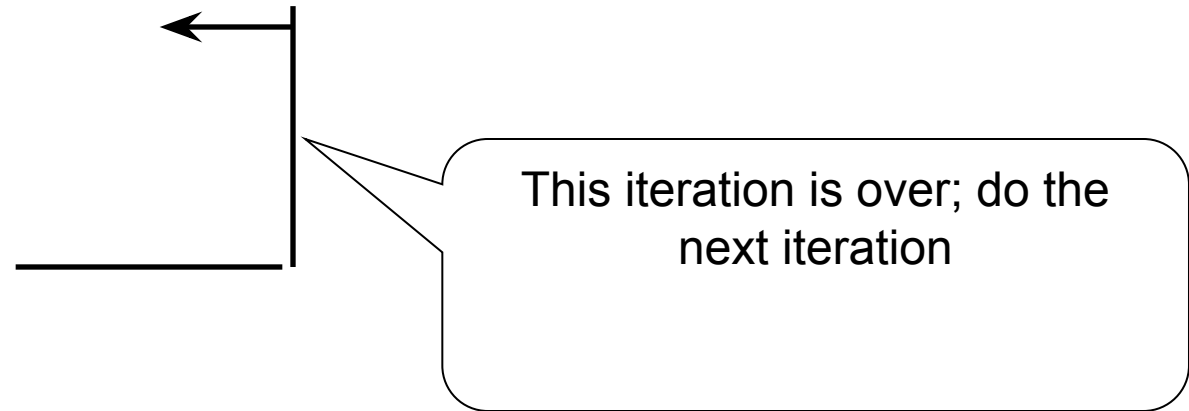
## The break command

```
while [condition]
do
 cmd-1
 break
 cmd-n
done
echo "done"
```



## The continue command

```
while [condition]
do
 cmd-1
 continue
 cmd-n
done
echo "done"
```



Example:

```
for index in 1 2 3 4 5 6 7 8 9 10
do
 if [$index -le 3]; then
 echo "continue"
 continue
 fi
 echo $index
 if [$index -ge 8]; then
 echo "break"
 break
 fi
done
```

## References

<http://www.freeos.com/guides/lsst/>

[http://linuxcommand.org/lc3\\_writing\\_shell\\_scripts.php](http://linuxcommand.org/lc3_writing_shell_scripts.php)

Based on Professor: Dr. Shu-Ching Chen, TA: Samira Pouyanfar/Hector Cen, Spring 2018

Bash scripting, Katherine Holcomb, UVACSE